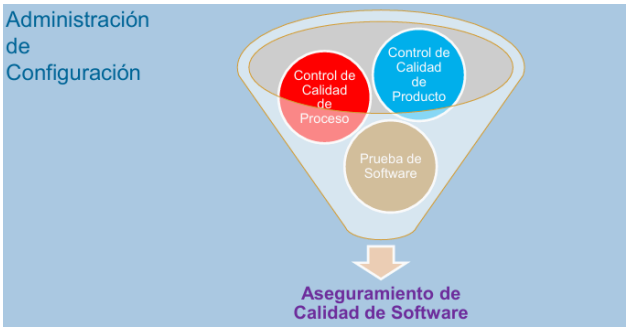
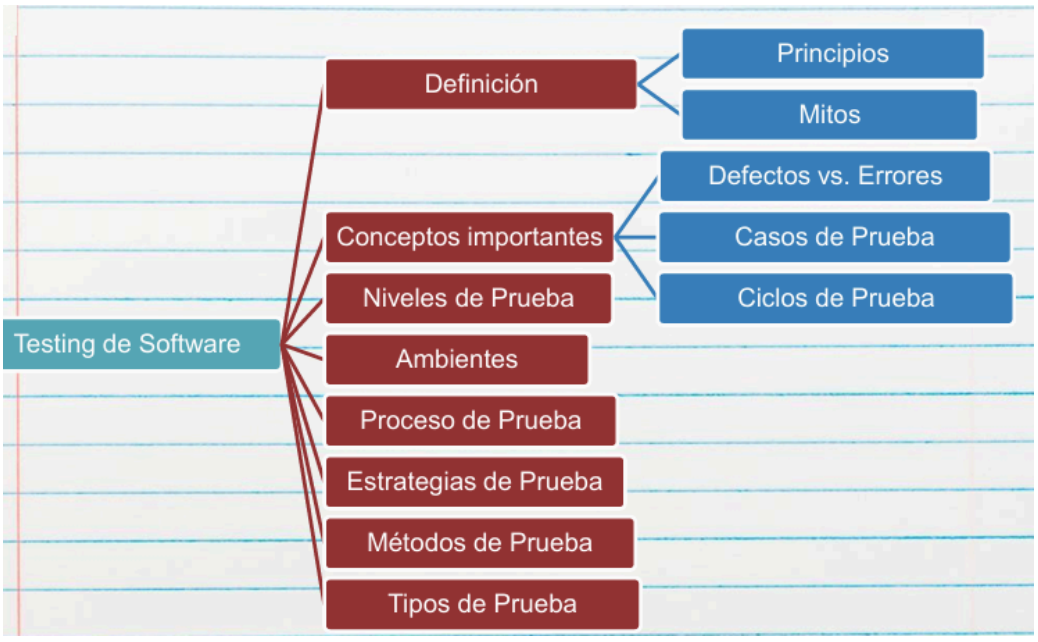
# SEGUNDO PARCIAL

## Testing




El testing es una disciplina de control de calidad centrada en la ejecucion de pruebas sobre el producto. Este NO inyecta calidad al final del ciclo de vida ni puede garantizar ausencia de errores, sino que su objetivo es hacer visibles defectos presentes bajo la hipotesis de que **efectivamente el software construido tiene defectos**.




Se dice que el testing es un proceso DESTRUCTIVO porque se parte de la hipótesis de que existen defectos y se diseñan pruebas para encontrarlos, con una actitud crítica frente al comportamiento del software. Una prueba que descubre defectos aporta información valiosa, aunque el testing también permite evaluar el cumplimiento de requisitos y generar confianza en el producto.

Cabe resaltar que QA (Quality Assurance) NO es lo mismo que testing por mas que en el mercado lo usen de sinonimos. QA es el conjunto de actividades planificadas y sistematicas aplicadas a lo largo de TODO el proceso de construccion de software para orientar el desarrollo hacia la calidad del producto y proceso, pudiendo generar que el testing FRACASE ya que no se encuentran DEFECTOS (lo cual no sería algo malo).

A su vez, también se debe definir a la calidad, la cual es la adecuacion del producto a las necesidades explicitas e implicitas del cliente, es decir, de sus necesidades y expectativas, cumpliendo con los requerimientos funcionales, no funcionales y estandares de desarrollo.

Los mitos del Testing son:

- El testing demuestra que el codigo no tiene errores → en realidad busca demostrar que si los tiene, revelando la mayor cantidad de errores posibles.
- El testing comienza luego de la codificacion → El testing debe integrarse en etapas tempranas del ciclo de vida, ya que la deteccion tempra ahorra esfuerzo y recursos
- El testing asegura calidad → Falso, como se menciono antes
- El tester es el enemigo del desarrollador

## *Error VS Defecto VS Falla*

Error: Equivocación humana (falla) cometida durante el análisis, diseño, programación o pruebas. Es detectado y corregido en la misma etapa.

Defecto: Imperfección introducida en un artefacto de software (requerimientos, diseño, código o documentación) como consecuencia de un error humano, el cual se traslada de una etapa a otra posterior en la cual se introdujo. Es DESEABLE que el comportamiento defectuoso sea REPRODUCIBLE, identificando condiciones y pasos para facilitar su diagnóstico y corrección, pero la reproducibilidad no determina su existencia.

Falla: Desviación observable del comportamiento esperado del sistema durante la ejecución. Un defecto puede causar una falla cuando se presentan ciertas condiciones; no todo defecto se manifiesta en cada ejecución.

## *Clasificacion: Severidad y Prioridad*

La severidad es el nivel de daño o impacto que el defecto causa sobre la estructura y funcionamiento del software, definida exclusivamente por el equipo de testing. La clasificacion siguiente no es taxativamente estricta ni un estandar, pero es una guia:

- Bloqueante: No permite la ejecucion del SISTEMA COMPLETO.
- Critico: Impide la ejecucion de una funcionalidad y no existe alternativa operativa para saciar la necesidad.
- Mayor: Pasa lo mismo que en el critico, pero existen alternativas para lograr el objetivo.
- Menor: Afecta aspectos secundarios sin impedir el funcionamiento del servicio.
- Cosmetico: Defectos esteticos.

La prioridad define el orden con el que se deben reparar los defectos segun el impacto comercial o necesidades del cliente. Esto es definido en las SLAs y el cliente o PO es quien determina la prioridad del defecto.

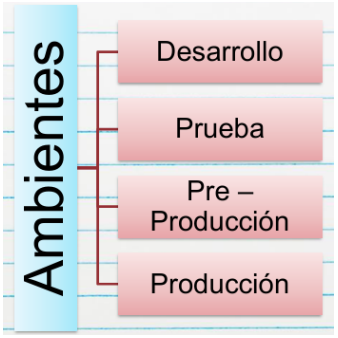
- Urgencia: Correccion inmediata sin postergar.
- Alta: Reparacion necessaria en el ciclo actual.
- Media: Corrección programada.
- Baja: Puede postergarse para versiones futuras.

La severidad y prioridad son metricas independientes, un defecto cosmetico puede tener una prioridad alta porque una es definida desde lo técnico y otra desde el negocio.

## *Niveles de Pruebas*

Los distintos niveles determinan el alcance y escala con la que se evalua el sistema, progresando desde componentes individuales hasta el producto completo.

- Pruebas unitarias: Son pruebas habitualmente ejecutadas por quienes desarrollaron el componente, enfocadas en verificar unidades aisladas, como métodos, funciones o clases. SUELEN ser de CAJA BLANCA debido al acceso al código y al uso de herramientas de desarrollo, pero también pueden diseñarse con técnicas de caja negra. Se apoyan en frameworks de automatización y prácticas como TDD (Test Driven Development). Los defectos encontrados suelen corregirse inmediatamente, sin necesidad de un registro formal.
- Pruebas de Integracion / Interfaces: Evaluan la interaccion entre interfaces de dos o mas componentes o subsistemas. Como resultado se obtiene la *buid* o paquete de componentes integrados. Se hace, en teoria en el Testing pero en la realidad se da en la produccion. Se puede autoamtizar con el CI.
- Pruebas de Sistema / Version: Llevada a cabo por los testers y equipo independiente de calidad (externo a los desarrolladores a fin de evitar sesgos), su foco es la validacion del sistema completo e integrado en un entorno que EMULA la produccion (un entorno con las mismas caracteristicas que el de produccion). Son pruebas que se ejecutan de manera sistematica, basandose en casos de prueba formales para garantizar reproducibilidad.
- Pruebas de Aceptación: Realizadas por usuarios, clientes o representantes del negocio (como el PO) para determinar si la aplicación satisface sus necesidades y los criterios de aceptación. Su foco es establecer confianza para utilizar el sistema, no encontrar defectos. Pruebas alfa: se realizan en un entorno controlado o de laboratorio, con participación de usuarios. Pruebas beta: se realizan en un entorno real de uso, con usuarios que evalúan el producto antes de su liberación general.

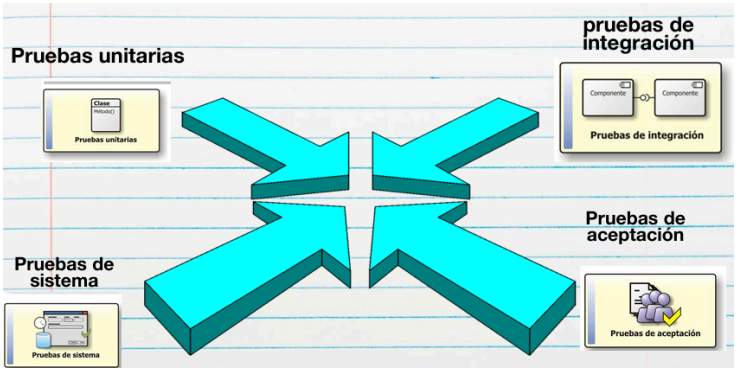


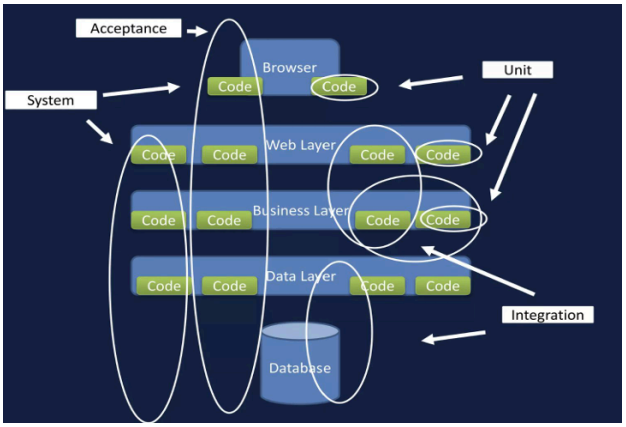

Los ambientes de desarrollo, pruebas, preproducción y producción permiten aislar actividades y gestionar las versiones mediante el plan de gestión de configuración. No es obligatorio disponer de un ambiente exclusivo para cada nivel de prueba. En particular, el ambiente de pruebas de sistema debe asemejarse al de producción tanto como sea posible. Como minimo son 3 ambientes (Desarrollo, Testing y Produccion)

El el ambiente de desarollose tiene el hardware y software necesario para desarrollar y desplegar el producto

En el ambiente de pruebas solo tienen acceso los testers y no los desarrolladores

Preproduccion e sun ambiente que emula las caracteristicas del ambiente productivo para comprobar que el producto funcionara una vez desplegado en produccion de manera correcta



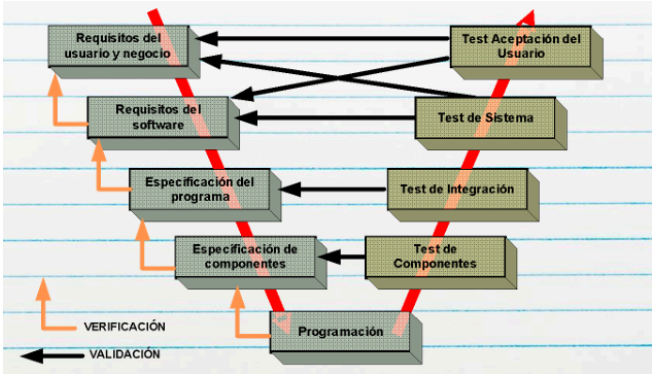




## *Validacion o Verificacion*

La verificacion consiste en preguntarse: ¿Estamos construyendo el sistema correctamente? Controla la correspondencia del producto contra las especificaciones técnicas e intermedias de diseño.

La validacion se pregunta: ¿Estamos construyendo el sistema correcto? Controla si el producto SATISFACE las necesidades y expectativas reales del cliente y su negocio.

Cabe aclarar que NECESIDADES son aquellas cosas que quedan EXPLICITADAS por el cliente, las expectativas son aquello que se ASUME.




Modelo V: Asocia cada fase de desarrollo con su correspondiente nivel de prueba:

- Requisitos de negocio - Pruebas de aceptacion
- Requisitos del sistema - Pruebas del sistema
- Diseño de arquitectura - Pruebas de Integracion
- Diseño de componentes - Pruebas unitarias

La curva de costo de deteccion temprana indica que a medida que se tarde mas en detectar un defecto en el ciclo de vida, mayor sera su costo de correccion.

## *Proceso de testing*

Las etapas son:

- **Planificacion y control**: Define la estrategia, alcance, riesgos, recursos, herramientas, ambientes y criterios de aceptacion. Se obtiene el plan de pruebas para controlar la toma de decisiones, revisar el resultado del testing y evaluar la cobertura del testing, debiendo contener los riesgos, objetivos, estrategias, criterios de aceptacion y recursos del testing..
- **Diseño (Identificacion y Especificacion)**: Identificación de condiciones de prueba, análisis de testeabilidad de requerimientos, diseño y priorización de casos de prueba y preparación de datos de prueba. Se obtienen los Casos de Prueba (Test Cases) y Datos de Prueba.
- **Ejecución**: Corre de las pruebas sobre la versión entregada (*build*), comparación de resultados obtenidos contra esperados y registro de desviaciones; puede ser automatizado con el CI/CD. Reporte de Defectos (Bug Report).

- **Evaluación y Reporte**: Análisis del cumplimiento de criterios de salida, verificacion de entregables y defectos corregidos, consolidación de métricas y evaluación de lecciones aprendidas.
- **Fin de pruebas**: Se deja de probar una vez se alcancen las consideraciones de defectos aceptados, generando un informe final.

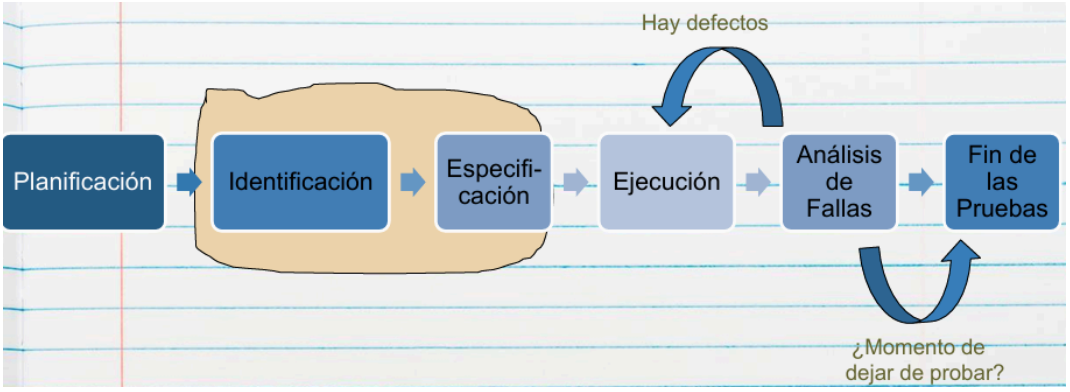
El testing exhaustivo es técnicamente imposible por la cantidad de combinaciones, el tiempo y el costo. Para determinar cuándo detener las pruebas se establecen CRITERIOS DE CORTE o salida, según los riesgos del sistema:

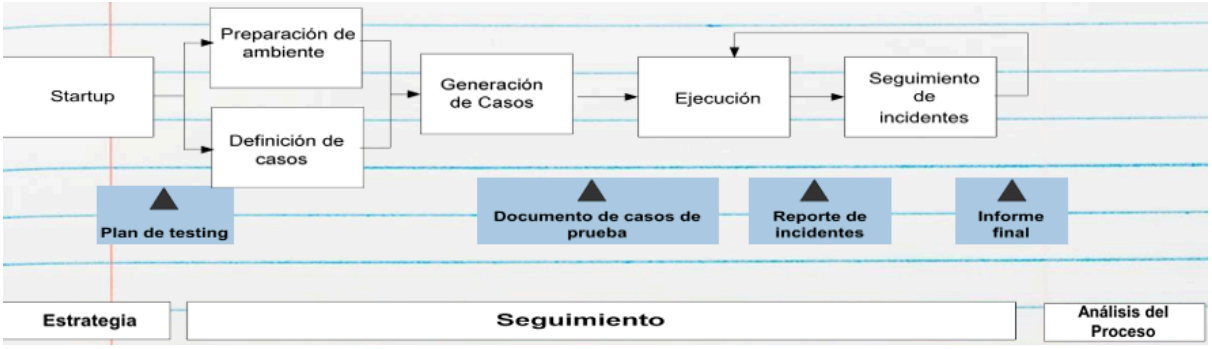
Criterio económico: detener el testing cuando el esfuerzo o costo adicional de las pruebas deja de justificarse frente al beneficio esperado y el riesgo residual.

Criterio de umbral de defectos: finalizar cuando los defectos abiertos se encuentran dentro del nivel aceptado (cantidad y severidad definidas previamente).

- Porcentaje de casos de prueba ejecutados y aprobados.
- Ausencia de defectos abiertos de severidad Bloqueante o Crítica
- Enfoque Good Enough: Nivel de riesgo aceptable donde las fallas remanentes no afectan la operación principal del negocio

En estos se determinan aquellos aspectos con mayor impacto y que deben ser testeados y con qué prioridad.




Los artefactos que se obtienen como salida son:

- Plan de pruebas
- Casos de prueba: Es la secuencia de pasos a seguir para lograr un resultado concreto y esperado, debiendo explicitar las condiciones de inicio y los datos utilizados.
- Reportes de incidentes o defectos
- Informe final

## *Estrategias de Prueba*

Las estrategias se seleccionan según objetivos, riesgos, requerimientos, tiempo y presupuesto. Según la clasificación enseñada por la cátedra, caja negra se presenta como DINÁMICO (comportamiento externo) y caja blanca como ESTÁTICO (estructura interna).

Se busca MAXIMIZAR los defectos detectados MINIMIZANDO los casos de prueba creados.

Caja negra — DINÁMICO: Evalúa el funcionamiento observable del software mediante ENTRADAS Y SALIDAS, sin conocer su estructura interna. Se ejecutan casos de prueba y se comparan las salidas REALES con las ESPERADAS según los requerimientos. La estrategia busca encontrar defectos maximizando la cobertura de condiciones relevantes con el menor costo y cantidad de casos posibles.

- Basadas en especificaciones:
    - **Partición de equivalencias**: Primero identifica las CONDICIONES EXTERNAS que intervienen en una funcionalidad (entradas, salidas, variables y restricciones que condicionan su ejecución). Luego divide los valores posibles de cada condición en CLASES DE EQUIVALENCIA: subconjuntos cuyos valores (condiciones externas) producen un comportamiento equivalente. Se identifican clases VÁLIDAS e INVÁLIDAS, provenientes de rangos continuos, valores discretos, selección simple o selección múltiple. Basta seleccionar un valor representativo de cada clase para diseñar los casos de prueba de esa condición.
      Ejemplo: si una edad válida es un entero entre 18 y 65 inclusive, las clases son: menor que 18 (inválida), entre 18 y 65 (válida) y mayor que 65 (inválida). Si el sistema también recibe letras, hay otra clase inválida correspondiente al tipo de dato incorrecto. Casos representativos: 17, 30, 66 y 'abc'.
    - **Análisis de valores límite**: Variante de la partición de equivalencias que selecciona los valores situados en los BORDES de cada clase, donde suelen encontrarse más defectos que en sus valores interiores. Para un rango entero inclusivo de 18 a 65, se prueban 17, 18 y 19 en el límite inferior, y 64, 65 y 66 en el superior; se compara cada resultado real con el esperado.
    - Procedimiento de partición de equivalencias según la cátedra: (1) identificar las condiciones externas, entradas, salidas y restricciones; (2) definir para cada condición las clases de equivalencia VÁLIDAS e INVÁLIDAS; (3) tomar un valor representativo por clase; (4) diseñar los casos de prueba con esos datos, precondiciones, pasos ordenados y resultado esperado; (5) considerar combinaciones relevantes entre condiciones. El propósito es MAXIMIZAR los defectos descubiertos con la MENOR cantidad de casos y costo posible.
      Importante: si distintos subrangos válidos tienen un tratamiento distinto (por ejemplo, diferentes porcentajes de impuesto), corresponden clases diferentes aunque todos sean datos admitidos. Los límites deben calcularse según el tipo de dato y si el extremo es inclusivo o exclusivo.
    - Otras técnicas: combinación por pares, tablas de decisión y grafos causa-efecto.
- Basadas en la experiencia:
    - Adivinanza de defectos: utiliza conocimiento previo sobre fallas frecuentes para proponer pruebas.
    - Testing exploratorio: combina aprendizaje, diseño y ejecución de pruebas durante la exploración del sistema.

Ninguna técnica es completa; conviene combinarlas según riesgos, tiempo y presupuesto.

Caja blanca — ESTÁTICO (según la clasificación de la cátedra): Analiza la estructura interna del software, el código y su lógica, para verificar cómo está construido. En el material de la cátedra se mencionan técnicas de cobertura estructural:

- Cobertura de caminos básicos: identifica los caminos independientes de la lógica del programa mediante su flujo de control y diseña casos de prueba que permitan recorrerlos.
- Cobertura de sentencias: busca que cada sentencia o instrucción ejecutable del código se ejecute al menos una vez durante las pruebas.
- Cobertura de decisiones: busca que cada decisión lógica (por ejemplo, un if) produzca tanto el resultado VERDADERO como el FALSO durante las pruebas.
- Cobertura de condiciones: busca que cada condición simple que integra una expresión lógica tome los valores VERDADERO y FALSO, independientemente del resultado de la decisión completa.
- Cobertura de decisión/condición: combina los criterios anteriores, buscando ambos resultados para las decisiones y para cada condición individual. Cobertura múltiple: busca probar todas las combinaciones posibles de valores VERDADERO y FALSO de las condiciones de una decisión.

Una cobertura alta no garantiza la ausencia de defectos.

## *Ciclo de Test*

Los ciclos de pruebas abarcan la ejecucion de la totalidad de los casos de prueba establecidos aplicados a una misma version del sistema a probar. Como resultado de este, se obtiene un reporte de prueba hecho. Los incrementos se cuentan desde el 0 en adelante, y en el mejor de los casos habrán 2 ciclos, el 0 y el 1 para correcciones (por el supuesto de que SI hay defectos).

## *Regresión*

Al concluir un ciclo de pruebas y reemplazarse la versión del sistema, debe realizarse una VERIFICACIÓN TOTAL de la nueva versión con los casos de prueba establecidos, para detectar defectos introducidos al corregir los anteriores. La automatización de las pruebas de regresión, por ejemplo mediante CI/CD, facilita volver a ejecutarlas sistemáticamente. (Definición conforme al enfoque presentado por la cátedra).

## *Casos de prueba*

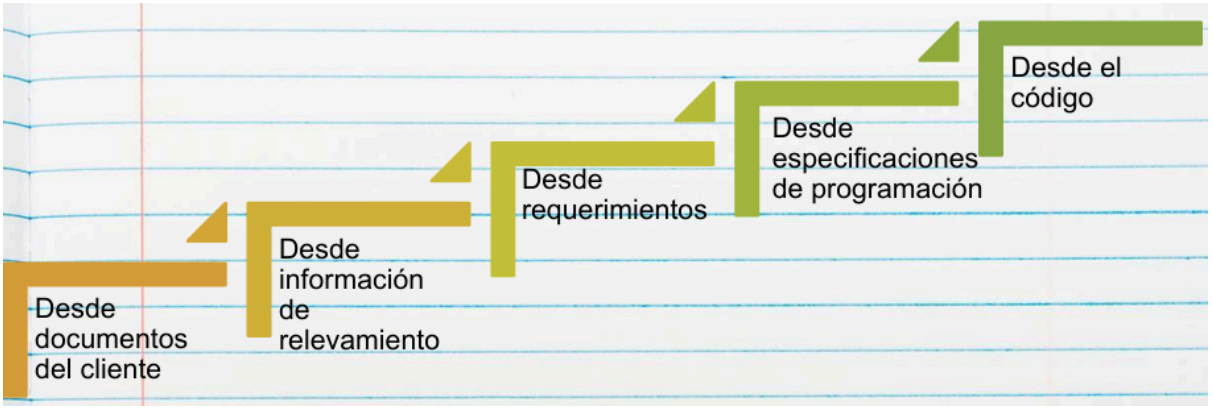
Unidades basicas de la actividad de testing, siendo el artefacto principal que se desarrolla durante la fase de Identificacion y Especificacion. Es el conjunto formal de condiciones previas, datos de entrada, secuencia de pasos e instrucciones, y resultados esperados cuya finalidad es la de descubrir la mayor cantidad de defectos posibles, evaluando las reglas de negocio de forma completa con el minimo esfuerzo y tiempo.

Los casos de prueba se diseñan durante el ANÁLISIS Y DISEÑO de testing y sirven como guía para su EJECUCIÓN. Derivan de documentos del cliente, información de relevamiento, requerimientos, especificaciones y código. Su diseño sistemático permite probar más condiciones relevantes con menos esfuerzo.

Los casos de prueba deben ser REPRODUCIBLES para poder catalogar y gestionar al defecto, como tambien deben ser REUTILIZABLES siempre y cuando los requerimientos no cambien (es decir, deben dar el mismo resultado bajo las mismas condiciones).

- Estructura de un caso de prueba:

- ID: Identificador único, generalmente numérico.
- Prioridad: Orden de ejecución según criticidad y capacidad de detectar defectos. Por ejemplo, el camino feliz o funcionamiento principal puede ser ALTA; combinaciones relevantes, MEDIA; y ciertas validaciones secundarias de formato, BAJA. La prioridad concreta depende del riesgo.
- Nombre: Título descriptivo de la condición que se probará.
- Objetivo: Funcionalidad, regla o condición que se busca comprobar.
- Precondiciones: Estado inicial y características del ambiente, expresados mediante datos CONCRETOS para poder reproducir el caso.
- Pasos: Operaciones ordenadas y numeradas, indicando quién actúa y qué hace (por ejemplo: «El usuario ingresa...»).
- Datos de entrada: Valores específicos utilizados durante la ejecución.
- Resultado esperado: Salida o comportamiento que debe producir el sistema, de acuerdo con los requerimientos.




- Ninguna tecnica es completa
- Las tecnicas atacan problemas distintos
- Es mejor combinar las tecnicas para complementar las ventajas
- Depende del codigo a testear
- Sin requerimientos es mucho mas dificil

## *Principios del testing*

- El testing muestra presencia de defectos, pero no puede demostrar ausencia de estos (que no se encuentren no garantiza que no existan).
- Es IMPOSIBLE que sea exhaustivo, inviable economicamente y por tiempo, para ello se aplican criterios de aceptacion para definir el alcance adecuado. (es por ello que deben definirse aspectos para considerar al testing como suficiente)
- El testing temprano detecta defectos en etapas tempranas, reduciendo exponencialmente los costos de correccion.
- Los defectos tienden a agruparse en un numero reducido de modulos, aplicando el principio de pareto 80/20 → 80% defectos en el 20% de funcionalidades
- Paradoja del pesticida: Si los mismos casos de prueba se ejecutan repetidamente, eventualmente dejarán de encontrar nuevos defectos. Para mantener la efectividad, los casos de prueba deben revisarse y actualizarse periódicamente.

- El testing es DEPENDIENTE del contexto, las estrategias y tecnicas se dependen de la naturaleza y criticidad del software.
- Falacia de la ausencia de errores.
- Un programador no deberia probar su propio codigo; una unidad de programacion no deberia probar sus propios desarrollos; el esfuerzo del testing debe planificarse
- El costo del testing esta entre un 30% y 50% (retrabajo) del valor del producto

## *Smoke test*

Conjunto breve de pruebas iniciales sobre una build para comprobar que sus funcionalidades principales están operativas y que no presenta fallas catastróficas antes de ejecutar pruebas más profundas. Puede repetirse cada vez que se recibe una nueva build. Su nombre proviene de la electrónica.

## *Tipos de pruebas*

Cabe aclarar que lo funcional y no funcional son los requerimientos que se prueban.

**Testing Funcional**: Evalúa qué hace el sistema, analizando funciones y reglas de negocio frente a las especificaciones. Son basados en requerimientos (verificar caracteristicas y especificaciones documentadas) y en procesos de negocio (flujos y escenarios de trabajo operativos).

- Prueba de Operacion Normal (testing funcional): Buscar las pruebas que pasan de las US, intencion de usar el sistema en los terminos que hay que usarlo. Son el camino feliz.
- Pruebas negativas: Relacionada a aquellas pruebas en donde el sistema deberia fallar, al uso indebido del software.
- Pruebas de proceso de negocio (Circuito): Se concatenan las funcionalidades de las US para validar el proceso completo. (Puede ser un solo caso de prueba o un conjunto de casos de prueba para cada US y la suma genera la prueba de PN).
- Documentacion: Que los manuales de usuario, de configuracion, sean consistentes con el funcionamiento del sistema

Los casos de prueba pueden servir para distintas pruebas, pero no en el mismo instante

**Testing No funcional**: Evalua cómo funciona el sistema, son dependientes del entorno donde el sistema esta instalado (rendimiento, seguridad, usabilidad, fiabilidad, etc.).

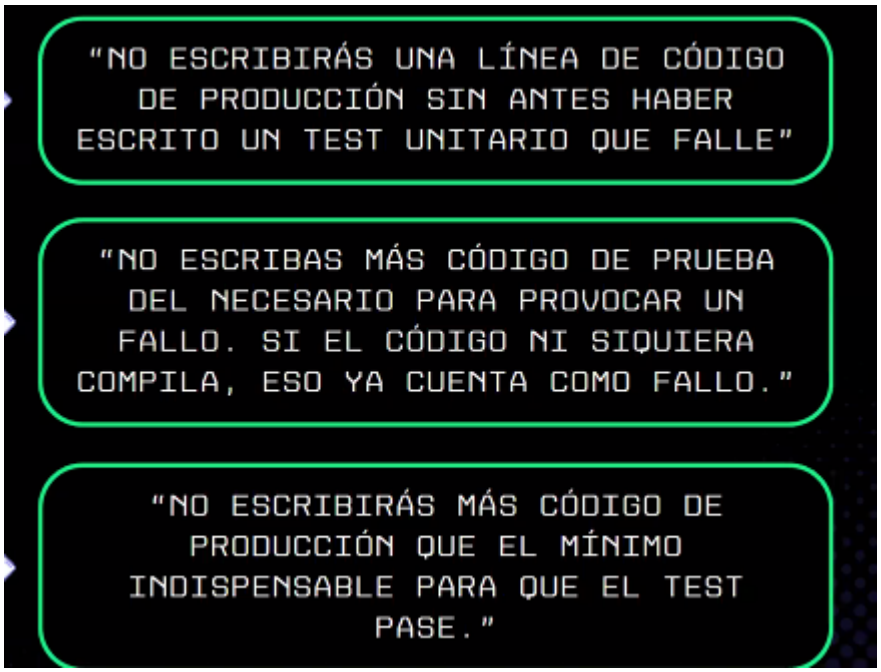
- Performance → Tiempo de respuesta en base a los recursos utilizados
- Escala completa → Probar la configuracion de hardware comprometida, el flujo normal de usuarios
- Stress → Probar que el sistema falle por someterlo mas alla de los limites comprometidos de funcionamiento, controlando cuanto tiempo lleva volverlo a poner operativo; hasta donde aguanta
- Carga → Concurrencia de usuario y uso de BD
- Usabilidad → Relacionadas con la experiencia de usuario ~ si el usuario le va a gustar el producto, enfocado al sentimiento del usuario
    - Accesibilidad → Uso permitido para personas con capacidades diferentes
- Matenimiento → Que tanto esfuerzo genera un cambio de requerimientos, el esfuerzo de evaluar un cambio

- Fiabilidad → Que tan confiable es el sistema (que tanto se pueden depender de sus resultados)
- Portabilidad → Controlar que tanto soporta el producto moverse de entornos (modelos de OS, de tecnologías, etc)
- Seguridad → Logica y fisica; ejemplos serian los permisos, datos a los que se puede acceder (fisica seria lo que puede hacer en persona; la logica son lo que se hace sobre el sistema)
    - Logica → Protege lo que se puede hacer dentro del sistema
    - Fisica → Protege a los recursos tecnologicos, equipos, instalaciones
- Configuracion → verifican que el software funcione correctamente bajo distintas configuraciones de hardware, software y entornos de ejecución. Busca detectar problemas de compatibilidad.
- Interfaces de usuario → Son mas tecnicas, relacionado a los componentes con los que el usuario puede interactuar, controlando que dichos componentes respondan correctamente

En los casos de prueba se pueden mezclar pruebas de requerimientos funcionales y no funcionales, pero no es óptimo

## *TDD: Test Driven Development*

- TDD (Test Driven Development): Práctica de desarrollo guiada por pruebas automatizadas. Primero se especifica un comportamiento mediante un test que falla; luego se escribe el código mínimo para pasarlo y finalmente se mejora el diseño sin alterar el comportamiento. El test guía la implementación y proporciona feedback temprano.
- Leyes del TDD:
- 1. No escribir código de producción antes de contar con una prueba automatizada que falle.
- 2. No escribir más código de prueba que el suficiente para producir un fallo; un error de compilación también puede contar como fallo.
- 3. No escribir más código de producción que el necesario para hacer pasar la prueba que falla.
- Objetivo: avanzar mediante ciclos cortos, mantener el código comprobable y favorecer un diseño simple; una alta cobertura puede ser consecuencia, pero no sustituye el diseño de buenas pruebas.




Ciclo TDD: RED → GREEN → REFACTOR.

RED: Escribir una prueba automatizada para expresar un comportamiento esperado y verificar que falla por el motivo correcto. Un caso de prueba puede estructurarse con precondiciones, pasos y resultado esperado (assert).

GREEN: Escribir el MÍNIMO código de producción necesario para que pase la prueba, sin anticipar funcionalidades no requeridas.

REFACTOR: Mejorar la estructura, legibilidad y diseño del código, eliminando duplicaciones SIN MODIFICAR su comportamiento observable; ejecutar las pruebas para comprobarlo.

Se repite el ciclo agregando casos progresivamente más exigentes. Mediante nuevos ejemplos y triangulación se pasa de soluciones específicas a implementaciones generales.

Patrones buenas practicas para hacer que el ciclo siga fluyendo y evite estancarse en la etapa de desarrollo.

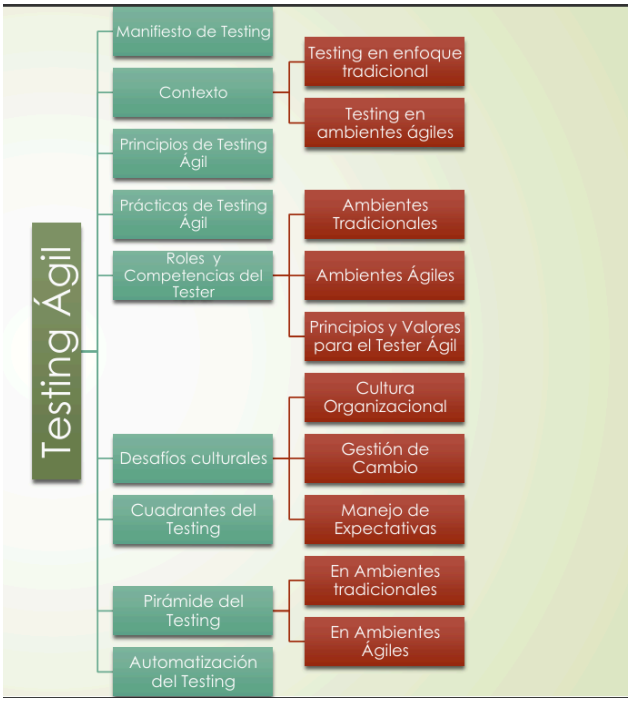
- Patrones red: Ayudan a definir qué se quiere hacer.
    - Starter test: Punto de partida, elegir la prueba mas trivial, facil y obvio.
    - One Step Test: Seguir con el siguiente test en la linea de complejidad, definir hitos para cumplir las metas
    - Explanation Test: Para etapa madura, permite debatir sobre el comportamiento esperado del sistema representado en un test.
    - Learning Test: Representar procesos de aprendizaje a traves de pruebas, entender funcionamiento del sistema para poder hacer integraciones.
    - Another Test: Evitar que permite anotar proximos tests como pendientes pero sin perder el foco de la tarea actual.
    - Regresion Test: Definir un test que replique los pasos de ejecucion de un problema para que no vuelva a ocurrir en el futuro. Replicar lo que sucedio para evitar que vuelva a pasar.
- Patrones Green
    - Fake it ‘til you make it: hardcodear la prueba para obtener el resultado.

    - Triangulación: Agregar ejemplos o casos de prueba distintos que obliguen a GENERALIZAR una implementación demasiado específica. Por ejemplo, luego de hacer pasar suma(2, 3) = 5 con un valor fijo, agregar suma(4, 6) = 10 exige implementar la suma real.
    - Obvious Implementation: si es algo muy obvio, se debe hacer la implementación correcta para cumplir con la prueba en todos los casos posibles. Funciona en problemas con complejidad sencilla.
    - One to many: Por medio de las iteraciones, permitir que se vayan agregando en los parametros colecciones.
- Refactor: Mejorar la estructura interna del software sin modificar su comportamiento observable. En sintesis, usar buenas practicas.
- Practicas de diseño:
    - Test list: Antes de desarrollar, hace rlistado de pruebas
    - Isolated test: Caracteristica, que los patrones sean los mas independientes del resto, pruebas unitarias
    - Test first: Primero escribir prueba y luego codigo
    - Assert Test: Arrancar del resultado (primero resultados, luego pasos, luego precondiciones). Ingenieria inversa
    - Test Data: La informacion que se debe usar debe ser lo mas sencilla y facil de entender
    - Evident Data: Relacion entre entradas y salidas sea lo mas obvia posible
- Otros grupos de patrones vinculados con TDD: patrones de testing, patrones de diseño y patrones xUnit (organización de fixtures, preparación, ejecución y verificación de pruebas).
- Ejemplo breve de TDD para una función sumar(a, b): RED → probar sumar(2, 3) = 5 y observar fallo; GREEN → devolver 5 para pasar ese caso; RED → agregar sumar(4, 6) = 10; GREEN → reemplazar el valor fijo por a + b; REFACTOR → mejorar nombres y estructura manteniendo todos los tests en verde.
- Diferencia con testing tradicional al final: en TDD el test antecede y dirige cada pequeño cambio de código, mientras que probar al final verifica una implementación ya construida.

# Testing de Software Ágil (Clase 09)

El testing agile es un enfoque en el que las actividades de prueba se INTEGRAN continuamente en el proceso de desarrollo, en lugar de concentrase en la fase final como en el tradicional. Busca obtener feedback temprano para detectar defectos rapidamente y contribuir a la calidad durante todo el desarrollo.

El cascada no puede usarse en el agil debido a que no permite cumplir con el DoD.



## *Pilares*

Automatizar todo lo que se puede automatizar, entendiendo que el tiempo sera un problema y restriccion → Permite la regresion

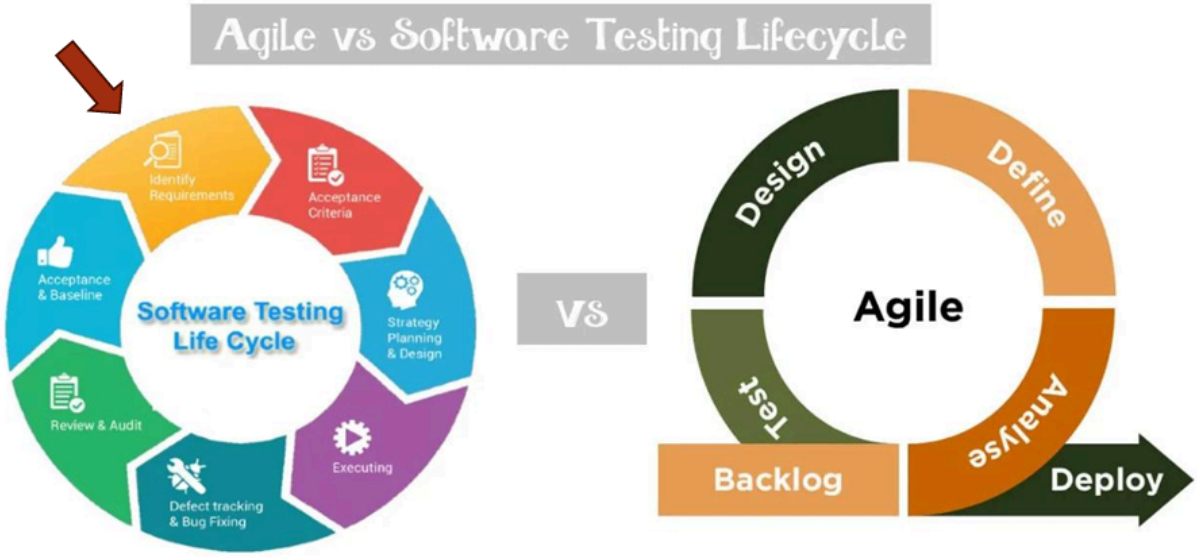
Se busca cambiar la idea del testing como destructivo

Se busca trabajar para la prevencion

Se busca entender las reglas de negocio (lo que se esta probando, apuntar a validar por sobre verificar). → Que genere valor

Conque todo el equipo es el responsable de la calidad es para evitar que toda la culpa recaiga en el equipo de testing




Se intenta desdibujar la linea de desarrollo y testing

## *Principios*

1. Testing se mueve hacia adelante en el proyecto: Significa que se comienza a ejecutar lo antes posible para detectar rapidamente los problemas.
2. Testing no es una fase: Se lleva a cabo de forma integra en el proceso de desarrollo, de forma continua

3. Todos hacen testing
4. Reducir la latencia del feedback: Reducir tiempo de retroalimentacion
5. Las pruebas representan expectativas: Es cómo se espera que funcione el sistema ante determinadas condiciones
6. Mantener el codigo limpio y corregir los defectos rapido: Evitar acumulacion de decretos y reducir retrabajo
7. Reducir la sobrecarga de documentacion de las pruebas: Evitar documentacion excesiva
8. Las pruebas son parte del Done: Considerar como terminada a una US si cumple los criterios de calidad y las pruebas establecidas
9. De probar al final ha conducido por pruebas: En lugar de construir primero y proar despues, las pruebas pueden utilizarse para guiar al desarrollo desde el inicio

Reducir la sobrecarga de documentacion → basada en la teoria de que el testing es la unica actividad que necesita el mayor detalle de documentacion → Casos de prueba, documentar requerimientos como pruebas; documentacion viva, todo detalle del testing en un solo documento que agrupa requerimientos y casos de prueba → Automatizar testing

### *Practicas Concretas*

- Pruebas utarias y de integracion automatizadas: Verificar componentes y sus interacciones frecuentemente
- TDD: Desarrollar codigo guiado por pruebas
- Regresion atuomatizada a nivel de sistema: Detectar defectos introducidos por los cambios
- ATDD (Desarrollo conducido por pruebas de aceptacion): Desarrollar a partir de las pruebas de aceptacion.
- Testing exploratorio: Explorar el producto e identificar comportamientos inestperados
- Versionado de pruebas junto al codigo: Mantener la correspondencia entre pruebas y versiones

### *TDD VS ATDD*

- TDD busca escribir primero las pruebas y luego el codigo minimi para superarlas para finalmente refactorizar (Red - Green - Refactor) → Concentrado en lo TECNICO
- ATDD: Primero se definen las pruebas de aceptacion y luego se establece qué comportamiento debe satisfacer el sistema segun los requerimientos del negocio. → Se enfoca en las necesidades y expectativas del cliente, en lo que lo satisface.

### *El rol de los testers*

En los enfoques tradicionales, el tester recibe el producto una vez ha sido desarrollado; en los ambientes agiles, el tester se integra al equipo y partricipa durante todo el desarrollo, colaborando en la rpevencion y deteccion temprana de defectos. Entre sus responsabilidades recae:

- Participar en la planning y comprension de requerimientos
- Colaborar con desarrolladores y representantes del negocio
- Ayudar a definir criterios de aceptacion
- Diseñar y ejecutar pruebas, manuales y automatizadas
- Validar el Done de las historias
- Contribuir a la mejora de los procesos de testing

## *Cambios culturales*

Cultura organizacional → Promover colaboracion entre testers, desarrolladores y negocio sobre el trabajo aislado

Gestión del cambio → Adoptar progresivamente practicas, responsabilidade sy formas de trabajar

Manejo de expectativas → Alinear lo que los interesados esperan del testing con las posibilidades, riesgos y tiempos del proyecto

## *Cuadrantes*

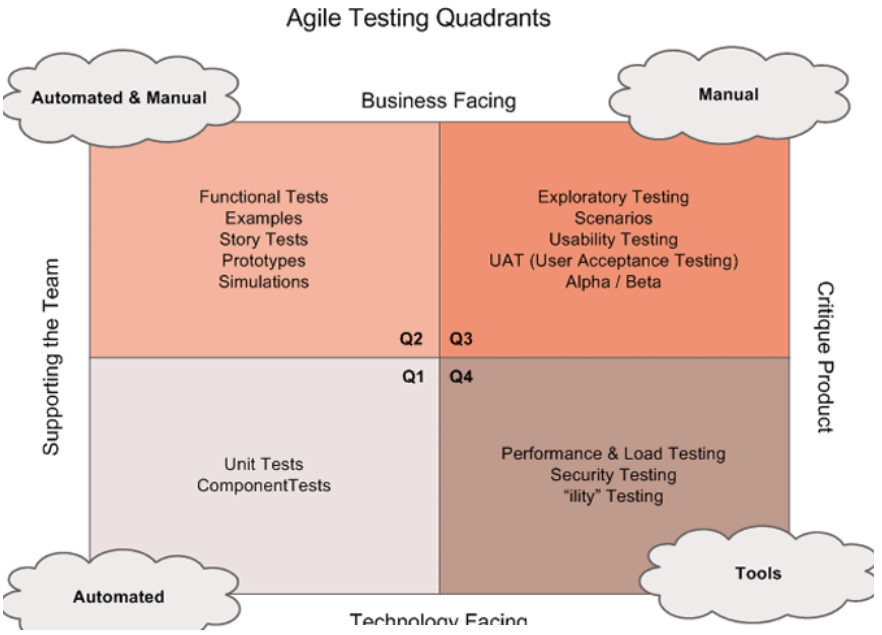
Los cuadrantes permiten clasificar las pruebas segun su proposito y orientacion, organizando los tipos de prueba y su momento de ejecucion.

Q1: comprobar aspectos tecnicos para ayudar al desarrollo

Q2:Comprobar comportamiento esperado para ayudar al desarrollo

Q3: Evaluar producto desde la experiencia del usuario

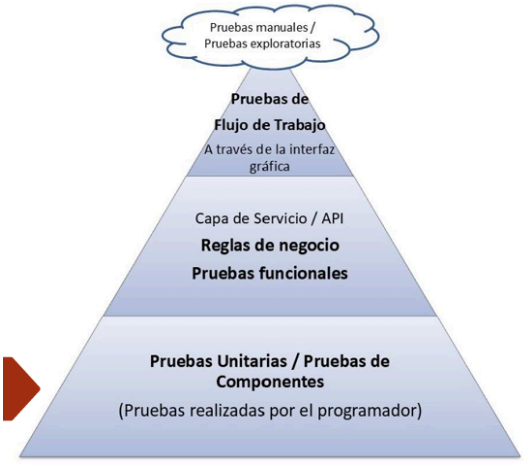
Q4: Evaluar caracteristicas tecnicas del producto (No funcionales)




## *Piramide de testing agil*

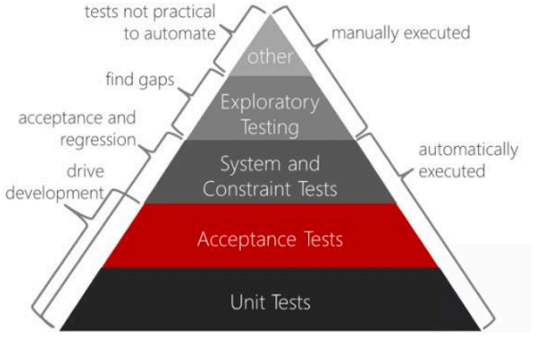
En el tradicional, se pone mas enfasis en las pruebas en la punta de la piramide; en el agil se busca poner enfasis en la base para reducir el esfuerzo de las pruebas de funcionalidad; esto ya que las pruebas unitarias son rapidas y automatizables.

- Base: Pruebas unitarias, rapidas y automazables, verifican componentes individuales
- Nivel medio: Pruebas de integracion, comunicacion de interfaces
- Cima: Pruebas de flujo de trabajo, se realizan recorridos completos del sistema que requieren de mayor tiempo y mantenimiento.




## *Automatizacion del testing*

Uso de herramientas para ejecutar pruebas y comprobar resultados de forma automática, facilitando su repeticion. Algunos beneficios son: Rapida ejecucion, reutilizacion, mayor cobertura, mayor precision, mayor alcance y reduccion de costos.




Esto permite ejecutar pruebas frecuentemente y obtener retroalimentacion rapida, PERO NO elimina la necesidad de pruebas manuales o exploratorias

# Extensiones de SCRUM

Existen frameworks para escalar la cantidad maxima de integrantes en SCRUM para gestionar productos de mayor tamaño, los cuales son Nexus, scale scrum, less y SAFe.

## *Nexus*

Framework que consiste en roles, eventos, artefactos y tecnicas que vinculan el trabajo de aproximadamente tres a nueve equipos Scrum que trabajan en un unico Product Backlog en comun para crear un incremento integrado de producto “Terminado”. Un solo PO que recibe un unico producto. surge para lidiar la complejidad de tener varios equipos trabajando sobre el mismo Backlog, lo cual tiene las siguientes dependencias:

- Requerimientos: Alcances que pueden superponerse
- Conocimiento del dominio: Debe ser equitativo entre todos para evitar interrupciones

### Responsabilidades

- Nexus Integration Team: Responsable en asegurar que se produzca un incremento integrado al menos una vez en cada sprint, abarcando restricciones técnicas y no técnicas.
- PO: Responsable en maximizar el valor del producto y gestionar el backlog
- Scrum Master: Responsable en asegurar el marco de trabajo Nexus se entienda y promueva
- Uno o ma smiembros del Nexus Integration Team: Ayudan a adoptar practicas y herramientas para lograr entregar un Integrated Increment que cumpla el DoD.

### Eventos

- Nexus Sprint
- Refinamiento entre equipos: El Backlog se refina en equipo para reducir dependencias, buscando TRANSPARENCIA y separar tareas.
- Nexus Sprint Planning: Coordinar equipos Scrum de un Nexus con los representantes de cada equipo y el PO. Se obtiene el Objetivo del Sprint, el del Nexus, el backlog del nexus, y del sprint.
- Nexus Sprint Review: Review entre todos
- Nexus Sprint Retrospective: Idem, retrospective con todos; se siguen haciendo las retrospective de cada equipo tambien. Consta de 3 partes:
    - Identificar problemas principales, dependencias e integracion
    - Retrospectivas individuales con cada equipo particular
    - Compartir resultados de las retrospectivas
- Nexus Daily: representantes de los Scrum Teams inspeccionan el progreso hacia el objetivo de Nexus y los problemas de integración, enfocada en la coordinacion e integracion entre los equipos. Las Daily Scrums de cada equipo complementan este evento, adaptando sus planes de trabajo para atender los problemas detectados.

### Artefactos

- Product Backlog
- Nexus Sprint Backlog: Objetivo del sprint nexus y sus elementos a llevar a cabo, resaltando dependencias y el flujo de trabajo de cada sprint. Es la suma de Sprint backlogs de cada team, y el objetivo nexus lo mismo pero con los objetivos.
- Integrated Increment: Incremento del trabajo integrado del sprint de cada equipo

## *Framework LeSS*

Framework para escalar Scrum que se aplica en productos de dos a ocho equipos. Propone un solo PO, un SC para uno o tres equipos y un unico Product Backlog. Si se trabaja con mas de ocho equipos se usa LeSS Huge

### Principios

- Scrum a gran escala es Scrum
- Mas con LeSS
- Pensamiento sistematico y Lean
- Control empirico de procesos
- Transparencia
- Mejora continua
- Centrado en el consumidor
- Enfoque de producto completo
- Teoria de colas

### Miembros

- SC: Enseña Scrum y LeSS, solo cumple el rol de master
- PO: Enseña Scrum y LeSS tambien, guia a todos a descubrir como pueden contribuir mejor a crear e producto
- Equipos

### Eventos

- Sprint Planning One: Atendida por el PO y los representantes de los equipos, seleccionan items tentativos
- Sprint Planning Two: Deciden cómo haran los items seleccionados, diseño y creacion del Sprint Backlog
- Daily: 15 min
- Review
- Restrospective: Particular por equipo
- Overall Restrospective: General entre todos
- Product Backlog Refinement: Refinamiento
- LeSS Sprint: a nivel de producto, no uno diferente para cada equipo. Todos al mismo tiempo

### Artefactos

- Incremento potencialmente entregable: Salida del Sprint, con la integracion del trabajo ed todos los equipos. deémde del DoD comun entre TODOS los equipos (aunque estos pueden tener un DoD mas detallado internamente, todos comparten una base)
- Product Backlog: un solo backlog, igual que en Scrum
- Sprint Backlog: Cada equipo tiene su propio sprint backlog,

### LeSS Huge

Se diferencia en:

- Tiene area de Product Manager
- Area de POs

- Area de Product Backlogs
- Area de Product Vision
- Mas de 8 equipos

# Filosofia Lean y Metodo Kanban

## *Filosofia Lean*

Lean es una filosofia de gestión orientada a MAXIMIZAR el valor entregado al cliente, minimizando el desperdicio, promoviendo la mejora continua de los procesos. Entiendase por desperdicio a todo aquello que consume recursos sin aportar valor significativo (por ejemplo, crear una funcionalidad que el cliente nunca usara).

### Principios

1. **Eliminar desperdicios (MUDA)**: Identificar y reducir las actividades que no aportan valor.
2. **Amplificar el aprendizaje**: Favorecer el conocimiento mediante experimentarcion, iteraciones cortas y feedback temprano. ~ Trabajo colaborativo en Agile
3. **Diferir compromisos**: Postergar decisiones irreversibles hasta el ultimo momento, cuando se dispone de suficiente informacion → NO significa demorar el proyecto, sino evitar tomar decisiones con gran grado de incertidumbre ~ Tomar decisiones tardias (estimar lo que hace falta ahora, detallar lo actual y postergar lo que se necesita despues)
4. **Entregar lo antes posible**: Reducir los tiempos en el que el cliente recibe valor, entregas frecuentes, para obtener la retroalimentacion. → No significa acelerar al proceso sino buscar hacer entregas parciales
5. **Dar poder al equipo**: Otorgar autonomia y responsabilidad, aprovechando conocimientos. Las decisiones tienen que ser llevadas a cabo por aquel que HAGA el trabajo
6. **Embeber la integridad conceptual**: Mantener coherencia en el diseño y construir calidad desde el inicio, tener un criterio en comun ~ Atencion continua a la excelencia tecnica, la calidad no se negocio.
7. **Ver el todo**: Optimizar el total por sobre optimizaciones locales que perjudican al rendimiento general ~ Entender el producto que se esta construyendo para entregar valor.

Para optimizar un proceso, primero se debe entender LO QUE SE BUSCA, es muy riesgoso querer eliminar desperdicios sin entender lo que se esta realizando.

### Desperdicio Lean tradicional

Como se menciono antes, el desperdicio es aquello que no aporta valor significativo, lo cual puede ocurrir por producir en exceso, defectos, tiempos de espera, falta de informacion, aplicar sobreingenieria, entre otros. Se clasifica en 2 tipos:

- Tipo 1 - Necesario: Aquel que no aporta valor directo pero si es necesario bajo condiciones actuales. Se busca MINIMIZAR, que tenga el menor impacto posible (por ejemplo, los managers o niveles de supervision en los procesos, los que generan burocracia).
- Tipo 2 . Puro: Directamente no aporta ningun valor ni es necesario. se busca ELIMINAR

7+1 Desperdicio Lean tradicionals

| Desperdicio Lean tradicional | Equivalente en software | Explicación y ejemplo |
|---|---|---|

| | | |
|---|---|---|
| Talento no utilizado (+1) | Talento no utilizado (+1) | No aprovechar habilidades, experiencia y creatividad del equipo. Ej.: ignorar las propuestas de mejora de los testers. |
| Inventario | Trabajo parcialmente terminado | Trabajo iniciado que todavía no genera valor entregado. Ej.: código desarrollado, pero sin integrar ni probar. |
| Movimiento | Cambio de contexto | Alternar continuamente entre actividades, perdiendo concentración y tiempo al retomarlas. Ej.: interrumpir desarrollo para atender incidencias. Perdida de productividad por tiempo de seteo. (PEOR FUENTE) |
| Esperas | Esperas y demoras | Trabajo detenido por dependencias o aprobaciones. Ej.: historia bloqueada a la espera de validación del cliente. El PO no tiene tiempo para las validaciones; no se aceptan los Pull request; que el cliente apruebe |
| Transporte ~ JIT intenta evitar gastos de stock | Cambio de tareas / transferencia de conocimiento | Traspasos entre personas o equipos que requieren coordinación y generan pérdida de contexto. Ej.: explicar repetidamente una tarea a distintos equipos. Agregar nuevas personas. Multitasking |
| Defectos | Defectos | Productos de trabajo con errores que requieren corrección y retrabajo. Ej.: código defectuoso que exige modificaciones y repetir pruebas. |
| Sobreproducción (saturar al mercado) | Características extras | Desarrollar funcionalidades no solicitadas o que no aportan valor. Ej.: implementar reportes que nadie necesita. |
| Sobreprocesamiento | Procesos extra | Ejecutar pasos, controles o procedimientos innecesarios. Ej.: aprobaciones redundantes para un cambio sencillo. |

### Relacion entre Lean y Kanban

Lean es la filosofia general mientras que Kanban aporta un metodo para visualizar, gestionar y mejorar el flujo de trabajo, incluyendo limites al trabajo en progreso (WIP).

## *KANBAN*

Kanban es un método (sistema de mejora) permite definir, gestionar y mejorar servicios que entregan trabajo del conocimiento. Su proposito es gestionar cómo fluye el trabajo a traves de un proceso, identificar problemas y mejorar progresivamente la entrega de valor al cliente. Significa "tarjeta de señalizacion" y su origen se vincula con el *just in time* de Toyota. Se centra en GESTIONAR EL FLUJO DE TRABAJO. En miniscula es la forma de visualizar el trabajo, el METODO es en MAYUSCULAS.

### 9 Valores Kanban

- **Transparencia**: Hacer visible el trabajo, su estado y los problemas
- **Equilibrio**: Equilibrar la demanda del trabajo con la capacidad del sistema
- **Colaboracion**: Trabajar de manera conjunta para mejorar el flujo
- **Foco en el cliente**: Priorizar la entrega de valor y necesidades del cliente
- **Flujo**: Favorecer el avance continuo y predecible del trabajo; el trabajo debe transcurrir en forma de flujo continuo, NO trabaja con PROYECTO (a diferencia de Scrum que trabaja con SPRINTS, con tiempo LIMITADO, TIMEBOX).
- **Liderazgo**: Promover iniciativas de mejora desde todos los niveles
- **Entendimiento**: Comprender como funciona el proceso y cuales son sus problemas
- **Acuerdo**: Establecer compromisos compartidos para introducir mejoras
- **Respeto**: Reconocer y valorar a las personas y sus responsabilidades.

### Principios Kanban

#### Principios de gestión del cambio

- Comenzar con lo que haces ahora: Analizar el proceso existente actualmente antes de intentar transformarlo. No obliga a cambiar inmediatamente la forma de trabajar. Lo que se hace ahora es lo que se mapea en el tablero y se buscan identificar desperdicios, haciendo cambios EVOLUTIVOS
- Respetar roles, responsabilidades y puestos actuales: Se parte de la organizacion existente sin imponer una nueva estructura
- Acordar la busqueda de la mejora continua mediante el cambio evolutivo: Introducir mejoras graduales, evaluando resultados
- Fomentar el liderazgo en todos los niveles: Cualquier integrante puede impulsar mejora, no solamente los responsables jerarquicos.

#### Principios de entrega de servicios

- Comprender y enfocarse en las necesidades y expectativas del cliente: El trabajo debe orientarse a la entrega de valor
- Gestionar el trabajo y permitir la autoorganizacion: Se gestiona el flujo y se deja que las personas organicen como realizar el trabajo
- Revisar periodicamente los servicios y sus politicas para mejorar los resultados entregados: Evaluar funcionamiento de sistema para mejorar resultados

La principal diferencia con el anterior es que la gestión del cambio explican como introducir y sostener los cambios, mientas que la entrega de servicios indica como gestionar el servicio que se entrega al cliente.

## Practicas Generales de Kanban (6 practicas)

### Visualizar el trabajo (LA PRACTICA!)

Representar el proceso mediante un tablero, haciendo visibles los estados, tareas y posibles bloqueos. Esto permite compartir informacion, facilitar la cooperacion y tomar decisiones rapidamente. ACLARACION! No es el mismo tablero que SCRUM, gestionan de forma distinta (SCRUM es TODO, DOING, DONE). NO existen dos tableros iguales, pueden ser similares. Cada PASO del proceso tiene su COLUMNA (cola)

### Limitar el trabajo en curso

Establecer una cantidad maxima de elementos que pueden encontrarse en progreso en un estado del proceso. Tener a todos los recursos ocupados puede ser perjudicial para el flujo, acumulandose colas, aumentando tiempos de espera y el trabajo tarda mas; se busca centrarse en el flujo y menos en mantener siempre ocupados a los trabajadores. En Scrum es a traves de la capacidad de un Sprint. Se usa para evitar los cuellos de botella, representado con un numero. Trabajar con piezas de trabajo de mas o menos el mismo tamaño, para evitar estimar (ya que lo considera un desperdicio).

### Gestionar el flujo

Buscar que el trabajo avance de forma fluida, predecible y sostenible desde que se solicita hasta que se entrega. Detectar cuellos de botella, administrar colas y medir el avance. La informacion de esta practica permite manejar las expectativas de los clientes, como tambien hacer predicciones y mejoras. Cada paso del proceso es una COLA

Administracion de colas:  Deja de empezar y empieza a terminar, hacer foco en lo que se necesita, analizandolo de derecha a izquierda (porque el WIP debe liberarse de la derecha hacia la izquierda para permitir el avance del trabajo)

### Hacer explicitas las politicas

Definir reglas claras y visibles para gestionar el proceso: Cuando entra una tarea, cuando puede avanzar, como se prioriza y que limites se aplican. Es decir, las politicas deben ser pocas, sencillas, bien definidas, visibles, aplicables y faciles de modificar por quienes prestan el servicio

### Establecer ciclos de retroalimentacion

Revisar periodicamente los resultados del trabajo y del proceso mediante reuniones y cadencias, para detectar problemas y tomar decisiones de mejora

### Mejorar colaborativamente, evolucionar experimentalmente

Introducir cambios evolutivos mediante experimentos acordados por el equipo. Si una mejora funciona, mantenerla, sino revertirla

| Cola de Producto | Análisis | | Desarrollo | | Listo para Build | En Testing | | En Producción |
|---|---|---|---|---|---|---|---|---|
| | En progreso | Hecho | En progreso | Hecho | | En Progreso | Listo para Despliegue | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |

60% en CU; 30% para mantenimiento; 10% Defectos. Las columnas HECHO sirven para ayudar al sistema PULL! → En caso de cuello de botella por WIP, las personas de la "derecha" pueden pasar a realizar pasos de la "izquierda".

Se debe poner WIP en la cola de producto PARA evitar que se acumulen funcionalidades infinitamente.

### Definir tipos de trabajo

La capacidad se asigna segun la demanda:

### Politicas explicitas

Las politicas explicitas son reglas acordadadas que determinan como debe gestionarse el trabajo dentro del sistema kanban, permitiendo que todos conozcan las condiciones de trabajo (inicia, desarrolla, prioriza y finaliza una tarea). Deben ser:

- Pocas, sencillas y bien definidas
- Visibles y acordadas entre las partes involucradas
- Aplicables en todo momento
- Fácilmente modificables por quienes prestar el servicio

Ejemplos son reglas para incorporar tareas al tablero, criterios para avanzar entre estados, los limites WIP, las clases de servicio y las cadencias de reuniones; el DoD; que cada US tenga su prototipo; DoR

### Clase de servicio

Permiten establecer diferentes politicas de tratamiento segun las necesidades y caracteristicas de trabajo:

- Expreso: Trabajo que necesita atencion urgente y puede interrumpir el flujo habitual. Puede exceder el limite WIP, poniendo en espera a los demas trabajos y, si es necesario, se realiza como entrega especial-
- Fecha Fija: trabajo que debe entregarse antes de una fecha, permaneciendo en cola hasta el momento que convenga iniciarlo, moviendolo a expreso si la fecha corre peligro
- Estandar: Trabajo habitual del sistema, se prioriza mediante un mecanismo definido basado en el valor del negocio. Puede usar un FIFO.
- Intangible: Trabajo cuyo beneficio o costo de postergación no siempre resulta inmediatamente visible. Beneficio preventivo o diferido

### Aplicación de Kanban

- Proceso: Modelar el proceso
- Trabajo: DEfinir la unidad de trabajo

- Limites WIP: Limitar WIP para ayudar al flujo de trabajo
- Politicas: Dan calidad
- Cuellos de botella y flujo: Mover recursos a los cuellos de botella
- Clase de servicio: Diferentes trabajos tienen diferentes politicas (ej, diferente DoD).
- Cadencia

### Modelar el proceso

Representar las etapas reales por las cuales atraviesa un elemento de trabajo, desde que ingresa hasta que se entrega. (Cola, analisis, desarrollo, listo para build, testing, produccion)

### Definir unidad de trabajo

Determinar qué elementos representar mediante tarjetas; por ej. CU, US, caracteristicas, defectos, tareas de mantenimiento, refactorizacion, actualizacion de infraestructura y solicitudes de cambio.

### Definir limites WIP

Cantidad maxima de elementos que pueden estar en progreso en cada estado, buscando controlar el trabajo simultaneo para mantener un flujo equilibrado y predecible

### Definir politicas de calidad

- Reposicion de trabajo: Cuando ingresan nuevas tareas, cuantas y quien lo decide
- Criterios de traccion: Cuando una actividad se considera terminada y puede avanzar al siguiente estado (no es el DoD)
- Limites WIP: Cuantos elementos pueden estar simultaneamente en un estado
- Clases de servicio: Reglas que se aplican a diferentes tipos de trabajo
- Cadencias de reuniones: Horarios, frecuencia y agendas de las reuniones
- Acuerdos de colaboracion: Otras reglas compartidas

### Cadencias de Kanban

Reuniones o actividades periodicas que permiten coordinar el trabajo y revisar el funcionamiento del servicio y promover mejoras. Como no hay un evento “final”, se debe establecer CUANDO hacer.

- Release o entregas: Coordiinar entrega al cliente del trabajo terminado
- Revision: Examinar el flujo, detectar problemas, evaluar mejoras.
- Planificacion: Determinar que trabajo incorporar y como organizarlo segun la capacidad y demanda.

| Cadencia | Ejemplo de frecuencia | Propósito |
|---|---|---|
| **Team Kanban Meeting** | Diaria | Observar y seguir el estado y flujo del trabajo (no de los trabajadores). ¿Cómo podemos entregar los elementos de trabajo más rápido en el sistema? ¿Hay capacidad disponible? ¿Qué debemos tomar a continuación? |
| **Team Retrospective** | Quincenal o mensual | Reflexionar sobre cómo el equipo gestiona su trabajo y cómo pueden mejorar |
| **Team Replenishment Meeting** | Semanalmente o a demanda | Seleccionar los elementos de la lista de trabajo para realizar a continuación |

### Metricas Kanban

Las metricas permiten evaluar cómo fluye el trabajo, detectar demoras, estimar tiempos de entrega y mejorar la previsibilidad del proceso.

- Lead Time: Vista del CLIENTE
- Cycle Time: Vista del EQUIPO

#### Lead Time - Tiempo de entrega

Tiempo que transcurre desde que entro la solicitud hasta que se entrego el resultado. Incluye los periodos en que el trabajo esta en cola, se desarrolla, prueba o permanece bloqueado. Tiempo que el cliente espera. Ritmo de entrega.

#### Cycle Time - Tiempo del ciclo

Tiempo transcurrido desde el comienzo del proceso de ejecucion de un elemento de trabajo hasta que finaliza. Ayuda a evaluar capacidad y rito de terminacion del equipo. No es lo mismo que Touch Time. Ritmo de terminacion, medido en dias de trabajo.

$$CycleTime = LeadTime - EsperaenBacklog$$

#### Touch Time - Tiempo de trabajo efectivo

Tiempo durante el cual el equipo trabaja efectivamente sobre un elemento, es el tiempo en que el trabajo fue realmente “tocado”, excluyendo tiempos de espera o de bloqueo. cuantos dias habiles paso en columnas de “trabajo en curso”

#### Eficiencia del ciclo del proceso

$$Eficiencia = \frac{TouchTime}{LeadTime} \times 100$$

La eficiencia permite identificar que proporcion del tiempo corresponde al trabajo efectivo frente al tiempo dedicado a esperas u otras interrupciones

### Metricas orientadas al servicio

Metricas que permiten evaluar qué nivel de servicio espera el cliente, qué capacidad tiene le sistema y qué compromisos se establecen

- **Expectativa de nivel de servicio (SLE)**: Nivel de servicio que el cliente espera recibir
- **Capacidad de nivel de servicio (SLC)**: Nivel de servicio que el sistema realmente es capaz de entregar
- **Acuerdo de nivel de servicio (SLA)**: Nivel de servicio acordado con el cliente
- **Umbral de adecuacion de servicio**: Nivel minimo por debajo del cual el servicio esulta inaceptable por el cliente

### Diferencias entre Scrum y Kanban

Entre ambos frameworks, no difieren en filosofia sino en la IMPLEMENTACION, las practicas de cómo llevar a cabo las variables

| Aspecto | Scrum | Kanban |
|---|---|---|
| Tiempo | Sprint, timebox | Cadencias variables |
| Compromiso | Compromiso de trabajo por iteracion | Opcional |

| Metrica | Velocidad | Lead Time promedio |
|---|---|---|
| Equipos | Multufuncionales | Multifuncionales o no |
| Seguimiennto | Burndown | No hay |
| Limite WIP | Indirecto, capacidad | Explicito |
| Agregar trabajo | Se protege sprint goal | Puede, si hay capacidad |
| Estimaciones | Para planificar | Opcionales |
| Roles | PO, SC, Dev | No describe |
| Priorizacion | Product Backlog ordenado | Opcional |
| Tablero | Organizado por Sprint | Persistente, representa flujo continuo |
| Tamaño del trabajo | Se busca completar las US por sprint | No describe |

# Métricas en los diferentes enfoques (Clase 11)

Una metrica es un valor numero que representa el grado o la presencia de determinados atributos en un proceso, proyecto o producto, permitiendo medirlos objetivamente. Estas permiten controlar el avance, evaluar resultados y utilizar informacion para la toma de decisiones; pero la medicion tiene un costo en recopilar, procesar e interpretar los datos, ademas de que no todas las metricas aportan informacion util.

## *Dominios*

- Metricas de proceso: Cómo funciona el proceso de desarrollo, identificar oportunidades de mejora.
- Metricas de proyecto: Cómo progresa el proyecto respecto de sus objetivos, estimaciones y restricciones; estas se consolidan para crear metricas de proceso que sean publica para la organizacion.
- Metricas de producto: caracteriscitas del software construido, tamaño, defectos y cobertura.

## *Metricas en el enfoque Tradicional*

En el enfoque tradicional se utilizan PROCESOS DEFINIDOS y métricas para controlar y comparar los resultados reales con los planificados.

Las cuatro metricas basicas son:

- Tamaño del producto: Magnitud del software que se construira o construyo. ES LA UNICA QUE MIDE PRODUCTO.
- Esfuerzo: Trabajo necesario o realizado, horas-persona-lineales
- Tiempo: Calendario, duracion de las actividades o del proyecto (No de trabajo lineal)
- Defectos: Cantidad y caracteristicas de los problemas detectados

Para…

1. Desarrollador: Esfuerzo, duracion estimada y actual de una tarea; % de cobertura de pruebas unitarias; numero y tipo de defectos encontrados por dichas pruebas y por revision por pares.
2. Organizacion: Tiempo calendario, desempeño del esfuerzo y del presupuesto, precision de estimaciones y defectos presentes en las versiones entregadas
3. Equipo: Tamaño del producto, cumplimiento de hitós, tareas planificadas y completadas, distribucion del esfuerzo, estado de los requerimientos y defectos.

### Principio "Manténgalo Simple"

Más metricas no significa necesariamente una mejor gestión, se debe medir aquello que aporte informacion para la toma de decisiones, respondiendo a estas preguntas:

- ¿Aporta información adicional a la que ya tenemos?
- ¿Tiene un beneficio práctico?
- ¿Responde a lo que necesitamos saber?

## *Métricas en el enfoque ágil*

En este enfoque se trabaja con procesos EMPÍRICOS: las decisiones se apoyan en la observación de los resultados y en la adaptación continua. Regla de oro de la cátedra: «La medición es una SALIDA, no una actividad». Se mide lo necesario y nada más. Dos principios orientadores: entregar temprana y continuamente software valioso, y considerar al SOFTWARE FUNCIONANDO como principal medida de PROGRESO.

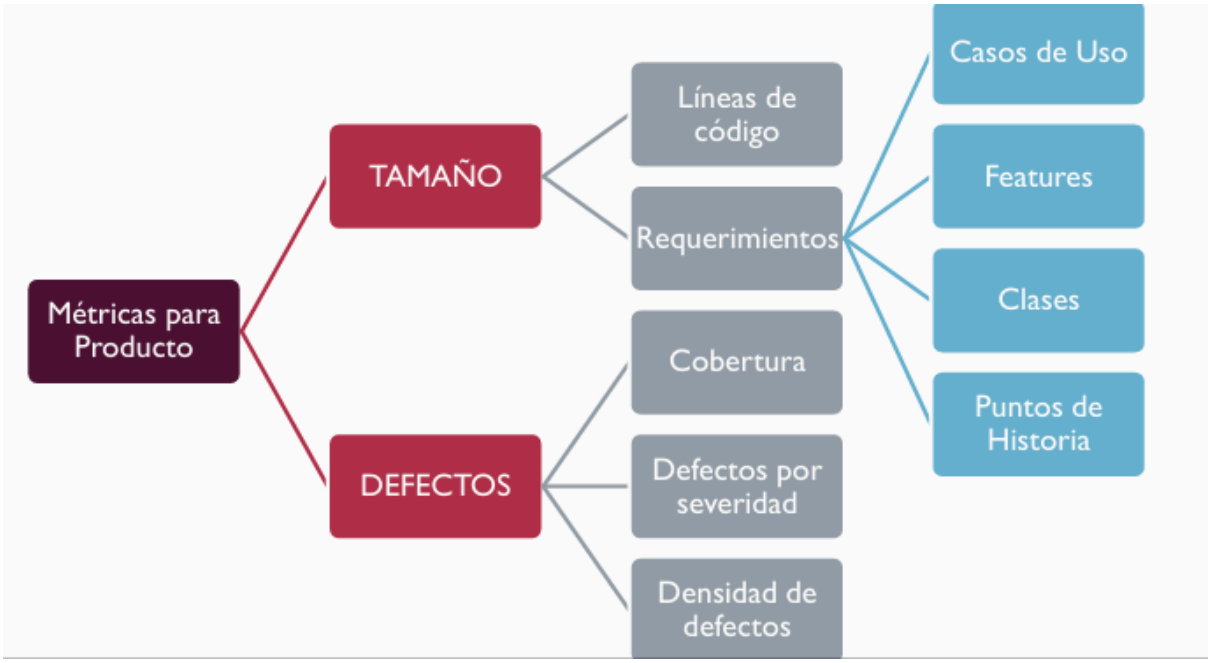
Las tres métricas son:

Velocidad: Cantidad de trabajo efectivamente COMPLETADO por el equipo durante un Sprint, generalmente expresada en Story Points. Es el resultado alcanzado y sirve como referencia para planificar.

Capacidad: Cantidad de trabajo que el equipo ESTIMA que puede asumir en una iteración según su disponibilidad. Puede expresarse en horas disponibles; NO equivale directamente a la velocidad medida en SP.

Running Tested Features (RTF): Cantidad de funcionalidades TERMINADAS, PROBADAS Y FUNCIONANDO, independientemente de cuántos Story Points represente cada una. Por ejemplo, si cinco historias están implementadas pero solo tres fueron probadas y funcionan, RTF = 3. Limitación indicada por Alex: NO considera la complejidad de cada funcionalidad. Su evolución permite observar si se están incorporando funcionalidades operativas.

*Metricas de producto*




### Tamaño

Mide magnitud del PRODUCTO a desarrollar. Puede contabilizarse con:

- Lineas de codigo
- Requerimientos
- Casos de uso
- Features
- Clases
- Story Points

### Defectos

Evaluan las caracteristicas de calidad del software a partir de los defectos y las pruebas.

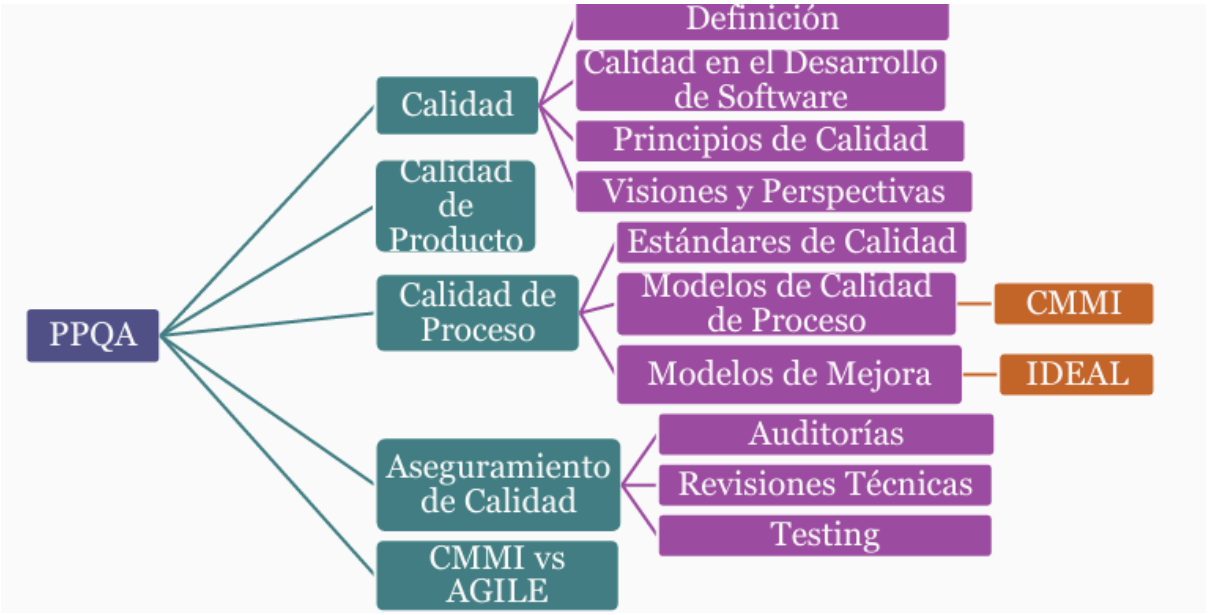
- Cobertura: Grado en que las pruebas abarcan los elementos o condiciones definidos.
- Defectos por severidad: Cantidad de defectos clasificados por gravedad.

Densidad de defectos: Cantidad de defectos en relación con una unidad de tamaño del software (por ejemplo, defectos por historia de usuario). Ejemplo: 12 defectos / 6 historias = 2 defectos por historia.

## Aseguramiento de Calidad de Proceso y de Producto (PPQA)

Entiendase por calidad a las caracteristicas de un producto o servicio relacionadas con su capacidad para satisfacer necesidades manifiestas o implicitas, por lo tanto es RELATIVA de las personas, circunstancias, necesidades y expectativas, las cuales pueden cambiar con el tiempo.

Se debe buscar satisfacer a los clientes, usuarios, gerencia, equipo de desarrollo, a los stakeholders en general. Por lo tanto la calidad depende de MULTIPLES factores.




## *Principios de la calidad (10)*

- La calidad no se inyecta ni se compra; se incorpora durante TODO el desarrollo
- Es un esfuerzo de todos juntos, una responsabilidad compartida
- Las personas son la clave, las competencias y capacidades del equipo son fundamentales
- Se necesita apoyo genencial, el compromiso de la direccion facilita los recursos y la continuidad de las iniciativas de calidad
- Liderar con el ejemplo, impulsar la calidad a traves de las practicas que la promueven
- No se puede controlar lo que no se mide, se deben usar metricas para conocer el estado y tomar decisiones
- Simplicidad, conviene implementar practicas comprensibles y sostenibles
- El aseguramiento debe planificarse, las actividades requieren de objetivos, recursos y planificacion
- Aumentar las pruebas no aumenta la calidad, no corrige las causas de dichos defectos
- Debe ser razonable para el negocio, el esfuerzo debe considerar las necesidades y restricciones del negocio.

## *Visiones (5)*

- Trascendental: Excelencia reconocible, aunque dificil de definir objetivamente
- Del usuario: Capacidad del producto de satisfacer necesidades
- De manufactura: Cumplimiento de especificaciones y procedimiento establecidos
- Del producto: Calidad asociada a sus caracteristicas y atributos
- Basada en el valor: Relacion entre beneficio y coste

## *Familia ISO/IEC 25000*

| Modelo | Qué evalúa |
|---|---|
| Calidad en uso — ISO 25010:2011 | La calidad experimentada por los usuarios al utilizar el producto. |
| Calidad de producto — ISO 25010:2011 | Las características de calidad del software. |
| Calidad de datos — ISO 25012:2008 | La calidad de los datos utilizados y gestionados. |

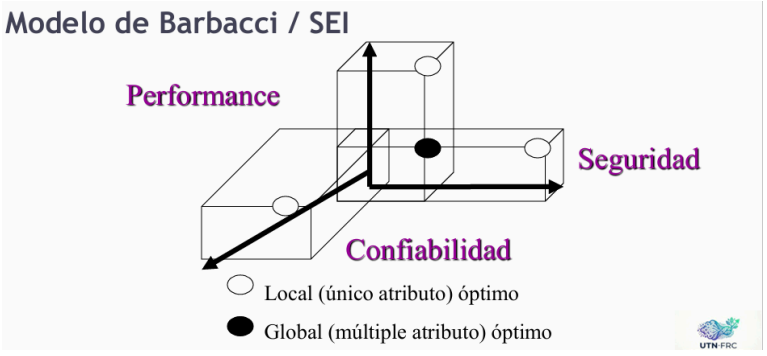
Definen la calidad del producto:

- Adecuacion funcional: El software proporciona funciones que SATISFACEN los requerimientos
    - Completitud funcional, correccion funcional, pertinencia funcional
- Eficiencia de desempeño: Que ofrece rendimiento adecuado considerando tiempos, recursos y capacidad
    - Comportamiento temporal, utilizacion de recursos, capacidad
- Compatibilidad: Puede coexistir e intercambiar informacion con otros sistemas.
    - Coefixstencia, interoperabilidad
- Usabilidad: Permite que el usuario comprenda, aprenda y utilice el producto
    - Reconocimiento de correccion, aprendizaje, operabilidad, proteccion frente a errores de usuario, estetica, accesibilidad
- Fiabilidad: Mantener su funcionamiento bajo ciertas condiciones establecidas y que puede recuperarse de fallos
    - Madurez, disponibilidad, tolerancia a fallos, capacidad de recuperacion,
- Seguridad: Protege la informacion y los accesos al sistem,a
    - Confidencialidad, integridad, no repudio, autenticidad, responsabilidad
- Mantenibilidad: Facilitar analizar, modificar, probar y reutilizar software
    - Modularidad, reusabilidad, analizabilidad, capacidad de ser modificado, capacidad de ser probado
- Portabilidad: Facilita adaptar, instalar o reemplazar el software en distintos entornos
    - Adaptabilidad, facilidad de instalacion, capacidad de ser reemplazado

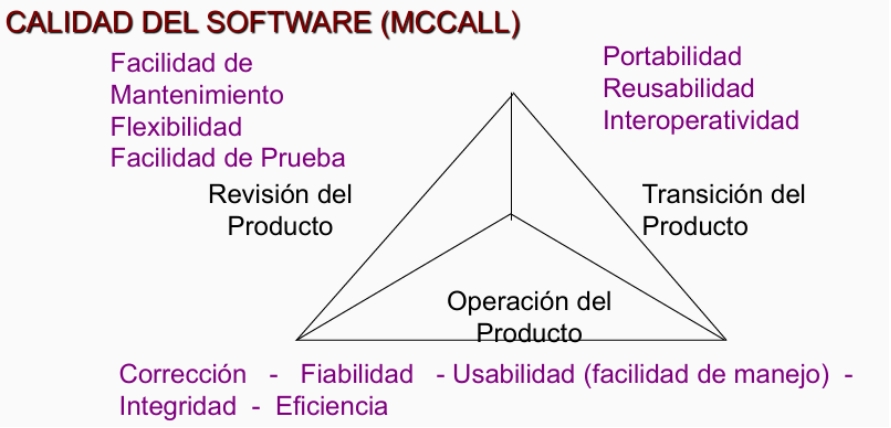
## *Otros modelos de calidad*

### Barbacci / SEI

Plantea la evaluacion de atributos de calidad y la relacion entre optimos locales y globales.



### Modelo de McCall




Tiene tres grandes agrupadores unos relacionados a la capacidad de verificación del producto (revisión del producto), otras con el producto en uso (operación del producto) y otras con los factores que influyen que el producto pueda crecer en el tiempo (transición del producto).

## *Calidad del proceso*

Se refiere a cómo se organiza y ejecuta el desarrollo de software, mediante actividades, metodos, practicas y procedimientos definidos. El proceso es un factor CONTROLABLE para mejorar la calidad.

Definir un proceso significa determinar:

- Quienes estan involucrados
- Qué Actividades a realizar
- Cuando deben ser ejecutadas
- Cómo se ejecutan

### Gestión de calidad de software

- Aseguramiento de calidad: Establecer estandares y procedimientos organizacionales de calidad
- Planificacion de calidad: Seleccionar y adaptar estandares y procedimientos al proyecto
- Control de calidad: Comprobar que se respetan los estandares y procedimientos

Existen 3 tipos de calidad

- **Calidad necesaria:** lo que realmente necesitan y esperan los interesados.
- **Calidad programada:** lo que se establece en los requerimientos y se planifica construir.
- **Calidad realizada:** las características que finalmente posee el producto desarrollado.

La gestión de calidad busca que las 3 coincidan, evitando la insatisfaccion del cliente como los desperdicios.

Ventajas

- **Menos retrabajo:** se evitan o descubren problemas antes de que se propaguen.
- **Menores costos de mantenimiento y soporte:** disminuyen las correcciones posteriores a la entrega.
- **Mayor satisfacción y confianza del cliente:** el producto responde mejor a sus necesidades.

- **Mayor previsibilidad:** los controles aportan información para gestionar riesgos y compromisos.
- **Mayor facilidad para evolucionar el producto:** un diseño y un código comprensibles facilitan los cambios.
- **Menor exposición a fallas:** se reducen riesgos para la operación y los usuarios.

Los costos son

| Tipo | Explicación y ejemplo |
|---|---|
| **Prevención** | Evitar que se introduzcan defectos. Ej.: capacitación y mejora de procesos. |
| **Evaluación** | Verificar la calidad del proceso o producto. Ej.: testing, revisiones y auditorías. |
| **Fallas internas** | Corregir defectos detectados antes de la entrega. Ej.: reparar un error durante las pruebas. |
| **Fallas externas** | Corregir problemas descubiertos después de entregar el software. Ej.: solucionar un defecto en producción y atender reclamos. |

Los estandares de producto son las caracteristicas; los del proceso son cómo se deben llevar a cabo las actividades

**Auditoría al Grupo de Aseguramiento de Calidad (GAC)**

Evaluación independiente destinada a verificar que el Grupo de Aseguramiento de Calidad cumple adecuadamente sus responsabilidades, aplica los procedimientos establecidos y ejecuta las actividades de aseguramiento previstas.

Su objetivo es comprobar la eficacia del propio grupo de calidad, en lugar de evaluar directamente el desarrollo de un proyecto o las funcionalidades del producto.

Ejemplo: verificar que el GAC realice las auditorías y revisiones planificadas, registre las desviaciones y efectúe el seguimiento de las acciones correctivas.

Reporte GAC: No deberia reportar al gerente de proyectos ni haber más de una posicion entre el gerente del primer nivel y el GAC. Cuando sea popsible, deberia reportar a un interesado

## Funciones

| Función | Explicación |
|---|---|
| Prácticas de aseguramiento de calidad | Desarrollar y disponer de herramientas, técnicas, métodos y estándares apropiados para realizar revisiones de calidad. |
| Evaluación de la planificación del proyecto | Comprobar que el proyecto contemple una planificación adecuada. |
| Evaluación de requerimientos | Evaluar la calidad de los requerimientos que guían el desarrollo. |
| Evaluación del proceso de diseño | Verificar que el diseño se realice siguiendo los procedimientos establecidos. |
| Evaluación de prácticas de programación | Comprobar la aplicación de estándares y buenas prácticas de programación. |
| Evaluación del proceso de integración y pruebas | Evaluar cómo se integran los componentes y cómo se realizan las pruebas. |
| Evaluación de planificación y control de proyectos | Evaluar los procesos utilizados para planificar, monitorear y controlar el proyecto. |
| Adaptación de procedimientos de calidad | Ajustar las actividades de aseguramiento a las necesidades de cada proyecto. |

**Concepto clave:** SQA no consiste solamente en probar el producto terminado, sino en evaluar la calidad de las actividades y procesos utilizados para construirlo.

## Proceso en la practica

### A. Definir procesos estándares

Establecer procedimientos que indiquen cómo deben realizarse las actividades del desarrollo.

Por ejemplo, definir cómo se conducen las revisiones técnicas o cómo se realiza la administración de configuración.

### B. Monitorear el proceso de desarrollo

Comprobar que los estándares y procedimientos definidos se estén respetando durante el trabajo.

Por ejemplo, evaluar si las revisiones de código se están realizando conforme al procedimiento establecido.

### C. Reportar los resultados

Informar sobre el proceso a la administración del proyecto y al responsable del software, permitiendo identificar desviaciones y adoptar acciones.

| Funciones de SQA | Calidad de procesos en la práctica |
|---|---|
| Establecen qué actividades deben evaluarse. | Determina cómo aplicar controles sobre los procesos. |
| Incluyen planificación, requerimientos, diseño, programación, integración y pruebas. | Incluye definir estándares, monitorear su cumplimiento y reportar resultados. |
| Permiten adaptar procedimientos a cada proyecto. | Evita aplicar estándares inapropiados para el contexto. |

## Aseguramiento de calidad

### Planificacion

Contiene:

- Productos de calidad deseados
- Atributos de calidad mas importantes
- Como se evalua la calidad
- Que estandares organizacional se aplicaran y si es necesario introducir otros

### Control de calidad

Controlar el desarrollo para comprobar que se siguen los estandares y procedimientos:

- Revisiones de calidad: Detectar problemas.
- Evaluaciones automaticas y mediciones: Controlar determinadas caracteristicas mediante herramientas y métricas

Objetivos: Inspecciones, revisiones de progreso, revisiones de calidad

### Modelos de mejora de procesos:

- SPICE: Software Process Improvement Capability Evaluation; ISO 15504, adaptacion para la evolucion de procesos de desarrollo de SW por niveles de madurez dividido en 6 niveles. Es un modelo DUAL, combina al SPICE + IDEAL. SPICE se utiliza para evaluar la capacidad de procesos de software; IDEAL guía las etapas de un proyecto de mejora. Pueden emplearse conjuntamente, pero cumplen funciones diferentes.

| Nivel | Estado |
|---|---|
| Nivel 0 - Organización inmadura | La organización no tiene una implementación efectiva de los procesos |
| Nivel 1 - Organización básica | La organización implementa y alcanza los objetivos de los procesos |
| Nivel 2 - Organización gestionada | La organización gestiona los procesos y los productos de trabajo se establecen, controlan y mantienen |
| Nivel 3 - Organización establecida | La organización utiliza procesos adaptados basados en estándares |
| Nivel 4 - Organización predecible | La organización gestiona cuantitativamente los procesos |
| Nivel 5 - Organización optimizando | La organización mejora continuamente los procesos para cumplir los objetivos de negocio |

- IDEAL: Da contexto para crear un proyecto cuyo resultado va a ser un proceso definido; modelo ciclico. Al inicio busca un sponsor, ya que las mejoras NO son criticas, debiendo tener un apoyo para cumplirse con exito. Se debe hacer un analisis de brecha (donde estamos y a donde queremos llegar)
    - Initiating: Iniciar mejora e identificar estimulo (necesidades, razones), establecer contexto, metas; se necesita del SPONSOR.

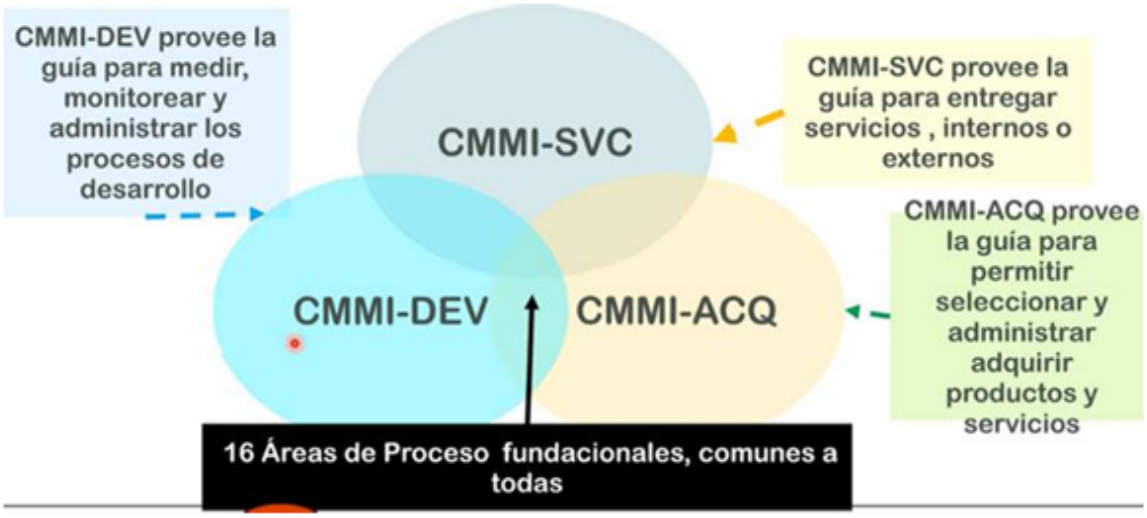
- Diagnosing: Diagnostricar situacion, problemas y oportunidades; riesgos y nivel de madurez
- Establishing: Establecer a estrategia, plan detallado de acciones, entregable y responsabilidades segun las prioridades.
- Acting: Actuar, efectuar cambios, reunir informacion
- Learning: feedback, ajustar, hacer que cada ciclo sea mas efectivo

## *Modelo CMMI*

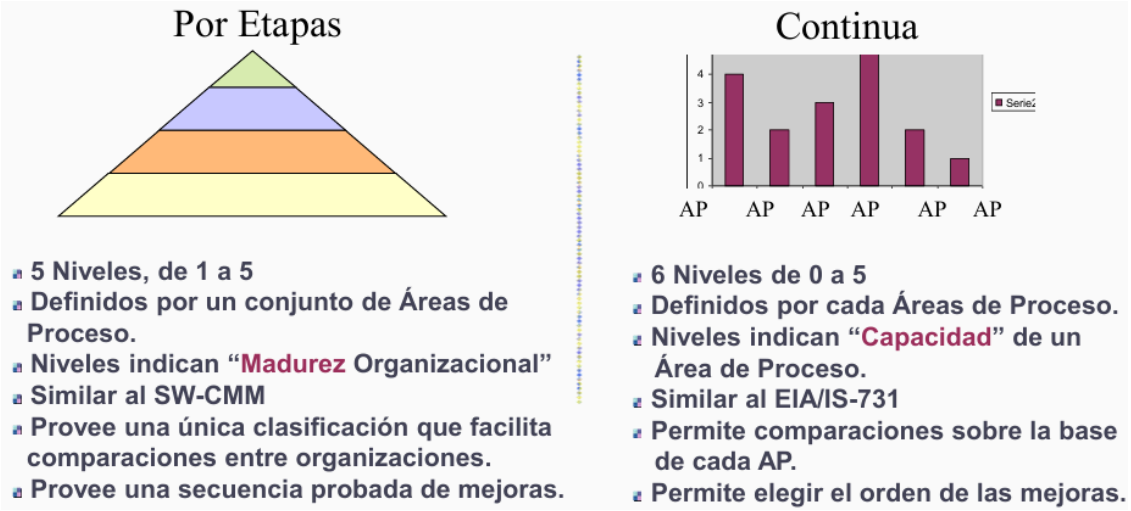
Capability Maturity Model Integration, modelo empirico orientado a evaluar y mejorar la capacidad y madurez de los procesos. Desarrollado por SEI, no es una norma ni certifica, solo evalua mediante profesionales autorizados (Lead Appraisers) y sirve de REFERENCIA.

**Constelaciones**

- CMMI-DEV: Medir, monitorear y administrar procesos de desarrollo; dos representaciones, por etapas y continua
- CMMI-SVC: Guiar entrega de servicios internos y externos
- CMMI-ACQ: Guiar la adquisición de productos y servicios; usado cuando la empresa DELEGA




**Representaciones**




**Por etapas**: Antecesor de CMMI, dividia a las organizaciones maduras (2 al 5) e inmaduras (1); mientas mas maduras, mas capacidad de lograr objetivos a bajo riesgo.

- **Inicial**: No disponen de ambiente estable para el desarrollo y mantenimiento de SW, resultados impredecibles de los proyectos.
- **Administrado**: Logran objetivos genericos y especificos de la madurez 2 → Proyectos cumplen con los requisitos de ser gestionados y que los procesos se planifican, realizan, miden y controlan. Razonable seguimiento de calidad
- **Definido**: Procesos bien caracterizados y entendidos; describen normas, procedimientos, herramienta y metodos; mayor grado de detalle en los procesos y realizacion de formacion personal. Metricas mas avanzadas, revision por pares.
- Cuantitativamente administrado: Metricas significativas de calidad y productividad, usadas de modo sistematico para la toma de decisiones y gestión de riesgos. SW de alta calidad; medidas analizadas estadisticamente
- Optimizado: Centrado en la mejora continua a traves de los aumentos y mejoras tecnologicas innovadoras




2 tipos de provedores: A → Cumnplen con el mismo nivel de calidad; B → no cumplen

**Continua:** Acreditar capacidad de la organizacion ante cada una de las areas de proceso de forma individual, buscando medir la capacidad de un proceso en particular en lugar de TODA la organizacion.

**Areas del proceso por nivel**

| Nivel | Áreas de proceso |
|---|---|
| 2 — **Administrado** | Administración de requerimientos; planificación de proyectos; monitoreo y control; acuerdos con proveedores; medición y análisis; **PPQA**; administración de configuración. |
| 3 — **Definido** | Desarrollo de requerimientos; solución técnica; integración del producto; verificación; validación; foco y definición de procesos organizacionales; capacitación; administración integrada de proyectos; administración de riesgos; análisis y resolución de decisiones. |
| 4 — **Cuantitativamente administrado** | Desempeño del proceso organizacional; administración cuantitativa del proyecto. |
| 5 — **Optimizado** | Administración del desempeño organizacional; análisis causal y resolución. |

**Roles y grupos**

Es importante que exista un responsable de cubrir las necesidades de cada rol o grupo (VER SI AGREGAR IMAGENES DE EJEMPLOS)

## *CMMI frente a Agile*

| CMMI | Métodos ágiles |
|---|---|
| Enfoque en medir y mejorar procesos. | Enfoque en responder a las necesidades del cliente. |
| Mejora organizacional y estandarización. | Mejora cercana al proyecto y al equipo. |
| Gestión del conocimiento mediante activos de proceso. | Gestión del conocimiento centrada en personas y comunicación. |
| Énfasis en predictibilidad y disciplina. | Énfasis en adaptación, creatividad y oportunidades. |
| Comunicación organizacional. | Comunicación interpersonal y colaboración directa. |

Se pueden combinar los enfoques, ya que el CMMI puede APOYAR, resultar NEUTRAL o entrar en CONFLICTO con ciertas practicas agile. NO SON NECESARIAMENTE INCOMPATIBLES.

CMMI y Agile no son necesariamente incompatibles. CMMI busca mejorar y evaluar los procesos organizacionales, mientras que Agile prioriza la adaptación y entrega de valor. Las prácticas de CMMI pueden apoyar, ser neutrales o entrar en conflicto con determinadas prácticas ágiles, dependiendo de cómo se implementen.

**Diferencias en valores esenciales**

- Prioridad: CMMI enfatiza medir y mejorar procesos; Agile, responder al cliente y reducir la sobrecarga.
- Comunicación: CMMI destaca la coordinación organizacional; Agile, la comunicación directa entre personas.
- Conocimiento: CMMI busca conservarlo en procesos y activos organizacionales; Agile destaca el conocimiento compartido mediante la colaboración.
- Personas: la filmina contrapone disciplina y seguimiento de reglas con creatividad y disposición a experimentar. Son énfasis ilustrativos: ambas formas de trabajo necesitan disciplina y creatividad.

Diferencias en características

- Mejora: CMMI enfatiza institucionalizar mejoras en la organización; Agile, inspeccionar y adaptar el trabajo del equipo y el producto.
- Resultados: CMMI busca consistencia y previsibilidad; Agile destaca aprovechar oportunidades y responder a cambios.
- Confianza del cliente: CMMI aporta evidencia de procesos gestionados; Agile enfatiza software funcionando y participación de los interesados.
- Organización del trabajo: CMMI destaca estructuras y responsabilidades organizacionales; Agile, autonomía y colaboración de los equipos.

Diferencias de enfoque y foco

- CMMI: énfasis organizacional y estratégico, procesos, medición, estabilidad y previsibilidad.
- Agile: énfasis en el producto, el cliente, la adaptación, la entrega frecuente y la rapidez de respuesta.
- CMMI indica objetivos y prácticas esperadas, dejando su implementación al contexto. Los marcos y métodos ágiles aportan formas concretas de organizar el trabajo.

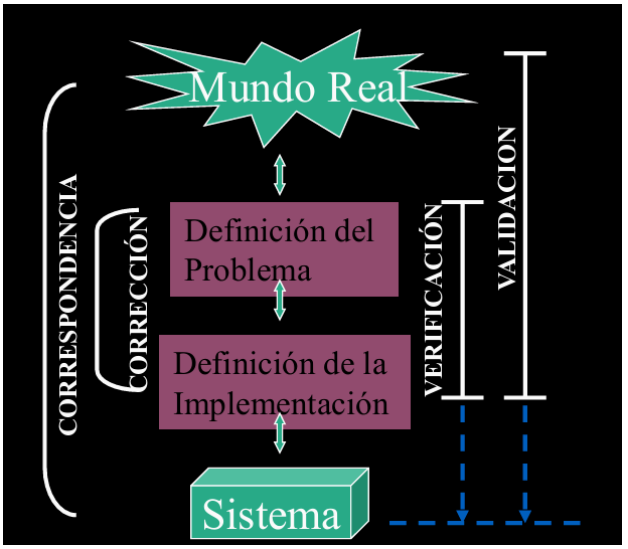
Estas diferencias no son exclusiones: Agile también mide, planifica y previene riesgos; CMMI también admite adaptación e innovación. Por eso, no conviene memorizar literalmente "Agile es solo cualitativo" o "gestiona riesgos de forma reactiva".

Similitudes

- Buscan organizaciones de alto desempeño.
- Ambos planifican y establecen reglas de trabajo.
- Se apoyan en experiencia y aprendizaje.
- Ninguno constituye una solución completa.
- Su aplicación debe adaptarse al contexto: ninguno sirve de la misma manera para cualquier proyecto.

# Revisiones tecnicas

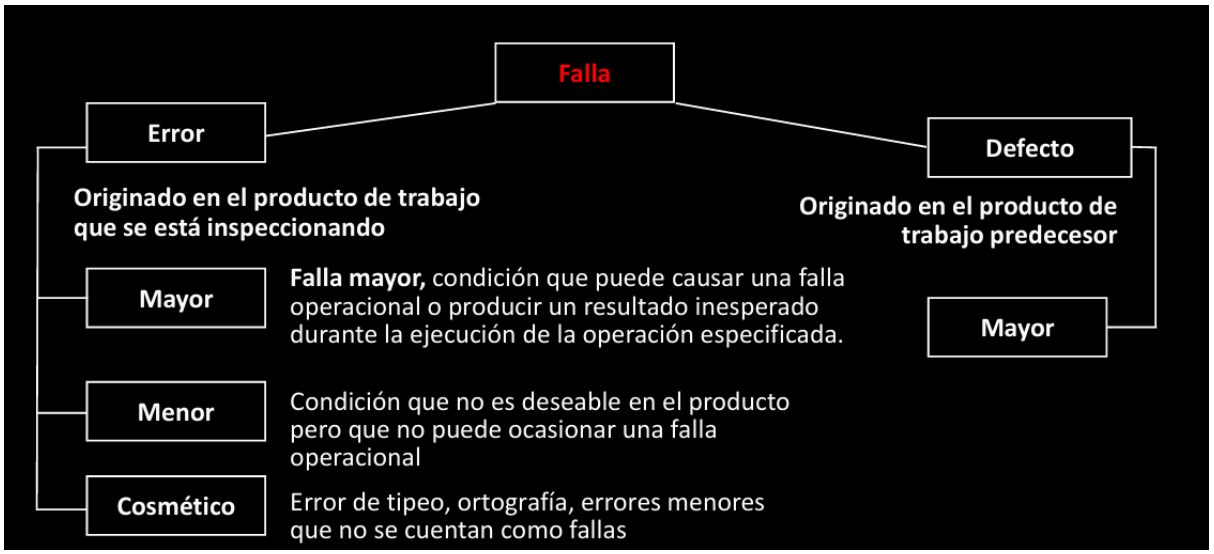
## *Verificacion y Validacion*



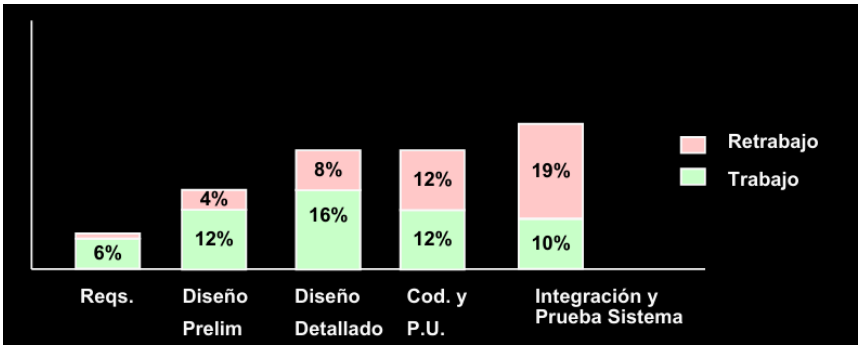

La V&V es un proceso que abarca todo el ciclo de vida del software, comenzando desde los requerimientos hasta las pruebas. Verificar consiste en controla lo que se esta construyendo, el medio; validar controla si se estra construyendo lo correcto en base a las necesidades del cliente.

El V&V se realiza tempranamente porque es un proceso que debe PLANIFICARSE, teniendo COSTOS ASOCIADOS que deben empezar durante las ETAPAS TEMPRANAS para garantizar calidad y evitar la correcciones de problemas detectados de forma tardia. Las FALLAS existen por ruido en la comunicaicon, por limitaciones de memoria y la complejidad inherente al producto

## *Clasificacion*




El retrabajo corersponde al 40% al 50% del desarrollo




## *Principios V&V*

Principalmente se busca PREVENIR defectos, APRENDER de estos y REDUCIR el retrabajo

- Prevencion e sla mejor cura
- Evitar es mas efectivo que eliminar
- La retroalimentacion enseña efectivamente
- Priorizar lo rentable
- Olvidarse de la perfeccion, porque no puede conseguirse
- Enseñar a pescar en lugar de dar el pescado

## *Revisiones Técnicas*

Proceso estatico de V&V cuyo objetivo es detectar defectos y corregirlos en etapas tempranas de desarrollo. Usadas cuando la calidad y retrabajo son criticos.

- Se pueden aplicar a cualquier representacion legible de SW
- PErmiten revisar productos incompletos
- No se ejecuta el programa
- Pueden detectar problemas que se pasan por alto
- Se complementan con las pruebas de SW

| Ventajas | Desventajas |
|---|---|
| • Pueden descubrirse muchos errores<br>• Pueden inspeccionarse versiones incompletas<br>• Pueden examinarse otros atributos de calidad | • Dificil introducir inspecciones formales<br>• Sobrecargan al inicio los costos, conducen al ahorro solo despues de la experiencia<br>• REquieren tiempo para organizarse y ralentiza el desarrollo |

Costos:

- Infraestructura: Entrenamiento, desarrollo (ajuste de plantillas, guias), implantacion de programas de medicion, herramientas de soporte
- Operacionales: Tiempo individual, grupal, y en completar los informes
- Adicionales: Calendario, material, datos

### Tipos de revisiones

Walkthrough: Autor conduce al participante a traves del producto. Estatica, pueden realizar comentarios y preguntas, pero sin control

Inspeccion: Revision formal y sistematica para identificar defectos mediante un procedimiento establecido; proceso controlado, con metricas

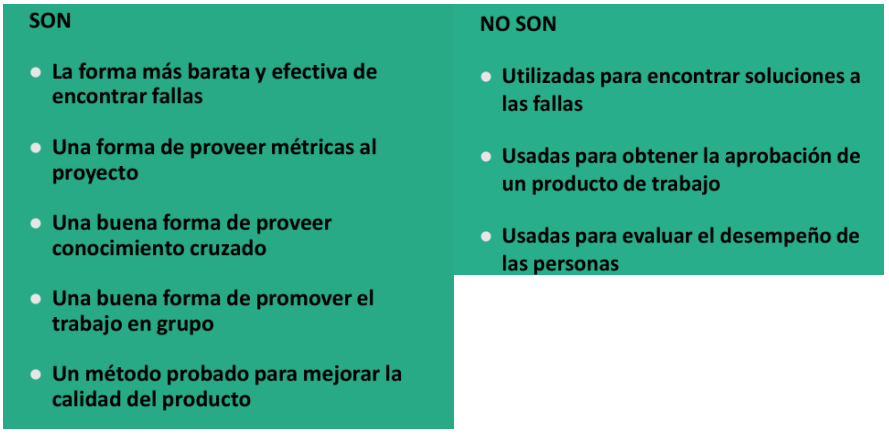
| Método | Objetivos Típicos | Atributos Típicos |
|---|---|---|
| Walktroughs | Mínima Sobrecarga<br>Capacitación de Desarrolladores<br>Rápido retorno | Poca o ninguna preparación<br>Proceso Informal<br>No hay mediciones<br>No FTR! |
| Inspecciones | Detectar y remover todos los defectos eficiente y efectivamente | Proceso Formal<br>Checklists<br>Mediciones<br>Fase de Verificación |

| Tipo de documento | Revisores sugeridos |
|---|---|
| Arquitectura o Diseño de alto nivel | Arquitecto, analista de requerimientos, diseñador, lider de proyecto, testers. |
| Diseño detallado | Diseñador, arquitecto, programadores, testers |
| Planes de proyecto | Líder de proyecto, stakeholders, representante de ventas o marketing, líder técnico, representante del área de calidad, |
| Especificacion de requerimientos | Analista de requerimientos, líder de proyecto, arquitecto, diseñador, testers, representante de ventas y/o marketing |
| Codigo fuente | Programador, diseñador, testers, analista de requerimientos |
| Plan de testing | Tester, programador, arquitecto, diseñador, representante del área de calidad, analista de requerimientos |

| Métricas Sugeridas | Fórmula |
|---|---|
| Densidad de defectos | Total de defectos encontrados / tamaño actual |
| Total de defectos encontrados | Defectos.Mayor + Defectos.Menor |
| Esfuerzo de la inspección | Esfuerzo.Planning + Esfuerzo.Preparación + Esfuerzo.reunion + Esfuerzo.Retrabajo |
| Esfuerzo por defecto | Esfuerzo.Inspeccion / Total de def encontrados |
| Porcentaje de reinspecciones | Cantidad Reinspecciones / Cantidad Inspecciones |
| Defectos Corregidos sobre Total de Defectos. | Esfuerzo.Inspeccion / tamaño actual |

### Inspeccion

Actividad de garantia de calidad de SW. Su objetivo es descubrir errores, verificar alcances, garantizar que el producto siga los estandares, que el producto sea uniforme, y que los proyectos sean manejables.




Proceso de Inspeccion

- Planificacion: Elegir participantes, preparar materiales y establecer el calendario. MAX 2Hs
- Vision general: Presentar el producto y el proceso de inspeccion
- Preparacion: Analisis individual para encontrar potenciales defectos.
- Reunion de inspeccion: El equipo recopila los posibles defectos y filtra falsos positivos
- Registro e informes: Decisiones sobre aceptar el producto, rechazarlo e informes de lo ocurrido
- Corrección: Autor Corregir defectos
- Seguimiento: Recopilar datos sobre correcciones

| Operación | Código | Documentos |
|---|---|---|
| Planificación | 15 minutos | 30 minutos |
| Vista previa | 500 LOC/h | 500 líneas de texto/h |
| Preparación | 100 LOC/h | 140 líneas de texto/h |
| Inspección | 125 LOC/h | 140 líneas de texto/h |
| Mejora del proceso | 30 minutos | 45 minutos |
| Tamaño máximo por inspección | 250 LOC | 280 líneas de texto |

| Rol | Responsabilidad |
|---|---|
| Autor | •Creador o encargado de mantener el producto que va a ser inspeccionado.<br>•Inicia el proceso asignando un moderador y designa junto al moderador el resto de los roles<br>•Entrega el producto a ser inspeccionado al moderador.<br>•Reporta el tiempo de retrabajo y el nro. total de defectos al moderador. |
| Moderador | •Planifica y lidera la revisión.<br>•Trabaja junto al autor para seleccionar el resto de los roles.<br>•Entrega el producto a inspeccionar a los inspectores con tiempo (48hs) antes de la reunión.<br>•Coordina la reunión asegurándose que no hay conductas inapropiadas<br>• Hacer seguimiento de los defectos reportados. |
| Lector | Lee el producto a ser inspeccionado. |
| Anotador | Registra los hallazgos de la revisión |
| Inspector | Examina el producto antes de la reunión para encontrar defectos. Registra sus tiempos de preparación. |

# Auditoria del Software (Clase 14)

La auditoria de calidad de software es una evaluacion independiente y objetiva de productos o procesos, cuyo proposito es verificar el cumplimiento de estandares, lineamientos, especificaciones y procedimientos. No solamente busca detectar defectos sino comprobar que el proyecto y productos respeten los lineamientos.

Se audita para:

- Obtener una evaluacion objetiva e independiente
- Identificar areas de posible insatisfaccion del cliente
- Comprobar cumplimiento con los compromisos
- Encontrar oportunidades de mejora
- Dar visibilidad a la gerencia sobre los procesos utilizados
- Evaluar cumplimiento del proceso de desarrollo
- Determinar la implementacion efectiva del proceso de desarrollo organizacional, del proyecto y las actividades de soporte

Como resultado se obtienen mejores productos, mayor satisfaccion y crecimiento del negoico

Inspeccionar es una tecnica para detectar defectos; Auditoria verifica objetivamente el cumplimiento de los criterios establecidos.

### *Tipos de auditorias*

- Auditoria de proyecto: Audita cumplimiento del proceso de desarrollo y consistencia de los productos a lo largo del proyecto. Ej: Comprobar que el codigo siga lo diseñado y este a los requerimientos → orientada al DESARROLLO
- Auditoria de configuracion funcional: Que el SW cumpla cumpla con las capacidades funcionales especificadas en la ERS (Y NO MAS!). Ej: Que haya correspondencia entre requerimientos, funcionalidades implementadas y matriz de trazabilidad → a los REQUISITOS
- Auditoria de configuracion fisica: Que lo construido y su documentacion sean consistentes por la TRAZABILIDAD. Ej: que los documentos sean consistentes con lo hecho. → a la DOCUMENTACION

### *PACS - Plan de Aseguramiento de Calidad de Software*

PACS vincula la planificacion del aseguramiento de calidad con la ejecucion de auditorias. NO confundir con plan de pruebas. Indica responsable de realizar auditorias. Incluye inspecciones, revisiones y pruebas.

Auditoria → Verifica consistencia del producto a medida que evoluciona a lo largo del proceso determinando que:

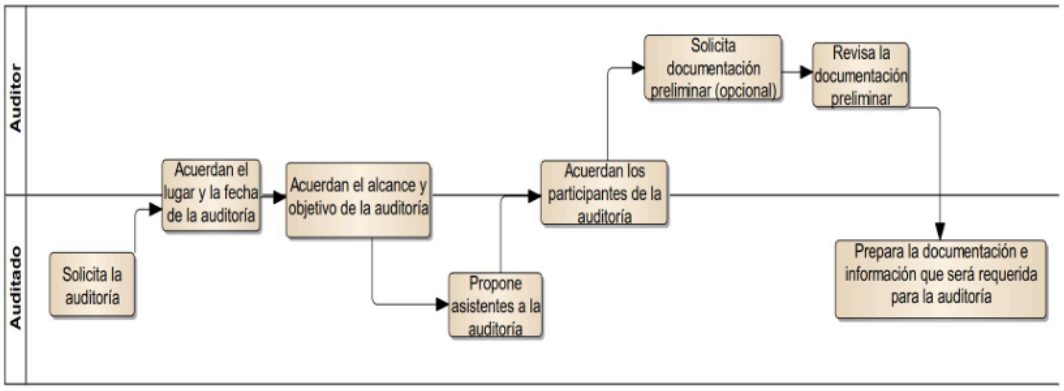
- Interfaces de HW y SW sean consistentes con ERS
- Req. Func. se validan en el Plan de V&V
- El diseño del producto, a medida que DDS evoluciona, satisface los req. func.
- Codigo consistente con el DDS (Documento de Diseño de SW).

### *Roles*

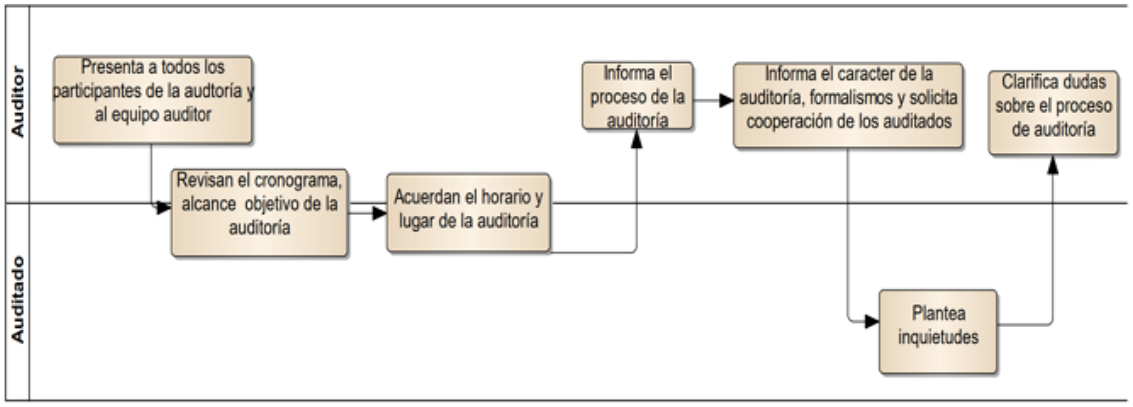
- Gerente SQA: Prepara plan de auditorias, calcula costos, asigna recursos y es responsable de resolver las no conformidaddes
- Auditor: Acuerda fechas y alcances, recopila evidencia, realiza auditoria, prepara el informe y sigue los planes de accion. DEBE SER DE FUERA DEL PROYECTO, puede ser una o dos personas
- Auditado: Proporciona evidencia, participa, responde al informe, propone acciones correctivas y comunica su cumplimiento.

### *Proceso de Auditoria*

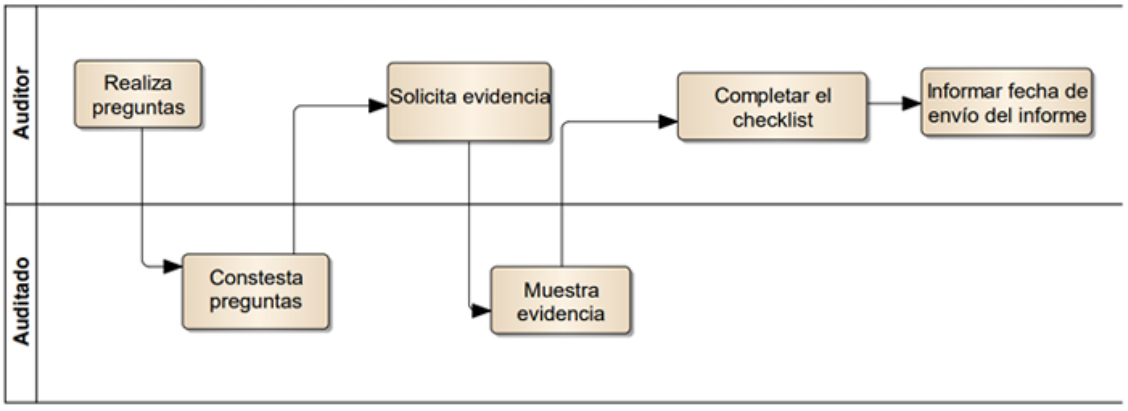
1. Preparacion y planificacion de responsabilidades: Definir fecha, alcance y objetivo, acordando los participantes. Definir metricas (esfuerzo por auditoria, cantidad de desviaciones, duracion de la auditoria, cantidad de desviaciones clasificadas)




2. Ejecucion: Auditor escucha, compara la documentacion con la evidencia
    a. Reunion de apertura: Informa como sera el proceso

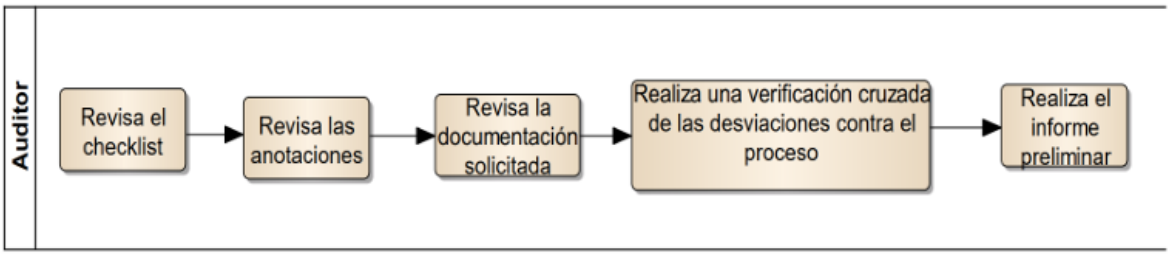



    b. Ejecución: Se responden las preguntas, se muestra evidencia y completa la checklist

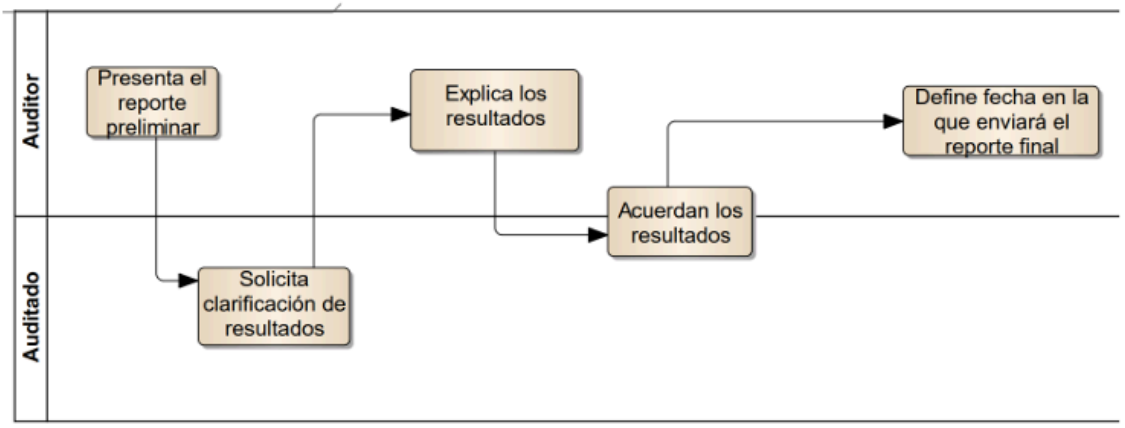



3. Analisis y reporte de resultados: Auditor realiza el reporte de resultados y el adutado analiza la respuesta. Puede estar o no deacuerdo con algunas practicas y dejarlo asentado. Evalua resultados, se hace una reunion de cierre y se hace una entrega del reporte final

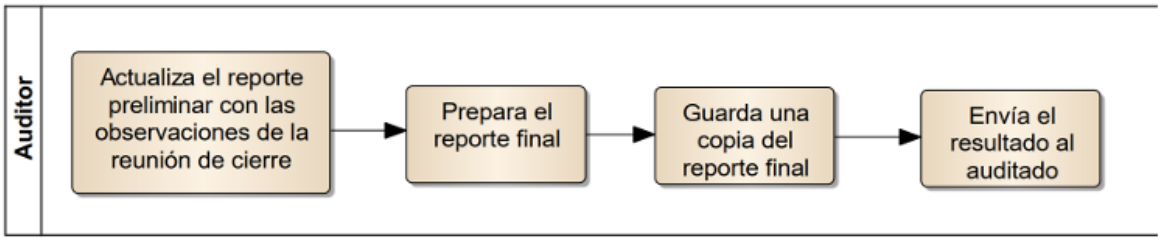
Evaluación de Resultados



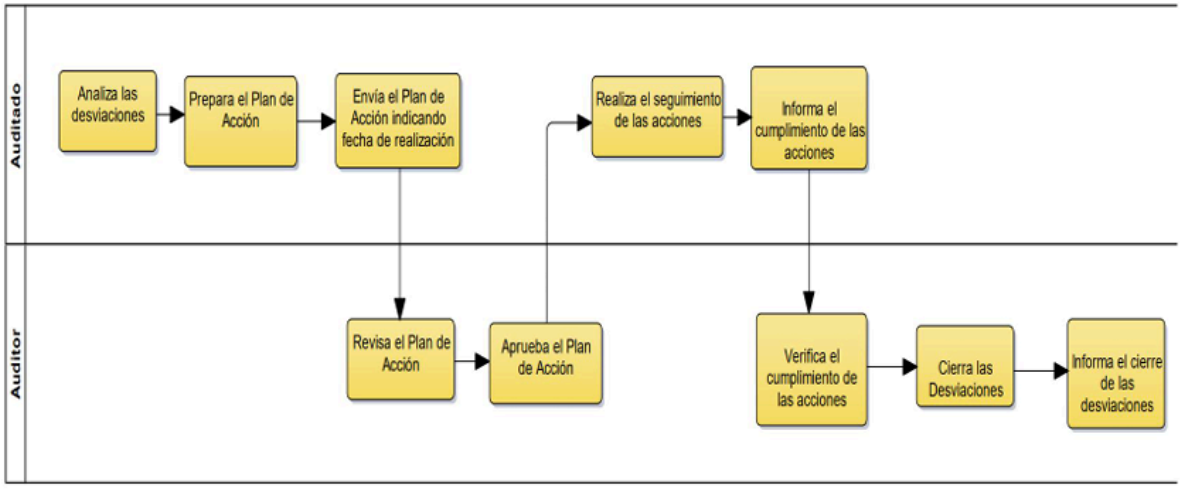

Reunión de cierre




Entrega de reporte final




4. Seguimiento: SE analizan las desviaciones y prepara un plan de accion que se envia al auditor para que lo revise y apruebe. En funcion del acuerdo entre auditado y auditor, puede que el auditor haga un seguimiento de las desviaciones que encontro hasta que las considere resueltas



### *Tecnicas y herramientas*

- Checklist: Preguntas para garantizar que, independientemente quien haga la auditoria, se haga foco en las cosas que se controlan. Deben garantizar lo minimo a contestar, pero se pueden hacer mas preguntas.
- Muestreo: Seleccionar muestra representativa de los productos/procesos
- Revision de registros: Analisis de documentos y evidencias
- Herramientas automatizadas: Soporte para recopilar y analizar informacion
- Tecnica del cuestionario

### *Resultados*

- Buenas practicas: Algo superador a lo definido, mejor que lo esperado
- Desviaciones: Cosas que no se hicieron como lo definido, clasificando como no adecuado (totalmente mal) o necesita mejora → Requiere plan de accion
- Observaciones: Practicas riesgosas, consideraciones que el auditor considera.

### *Reporte de auditoria*

Documento que informa de desviaciones que resultan en disconformidad o de los lineamientos, dejando asentado los resultados, observaciones y solicitud de planes de accion

### *Metricas*

- Esfuerzo por auditoria
- Cantidad de desviaciones
- Duracion de auditoria

### *Ejemplo de plan*

- *¿Existe un plan de proyecto?*
- *¿Está actualizado el plan de proyecto?*
- *¿Existe un responsable para cada actividad?*
- *¿Se han asignado recursos para las actividades de soporte?*
- *¿Están disponibles los planes para todos los involucrados?*

El auditor debe asegurarse que:

- Planes basados en requerimientos
- Actividades planificadas llevadas a cabo
- Todos se comprometan con los planes
- Cambios se hayan aprobado
- Decision de cambios documentadas
- Identificacion y comunicacion de los riesgos

## Gestión de Configuracion en Ambientes Agiles

Gestión que posibilita el seguimiento y coordinación del desarrollo, en lugar de controlar a quienes desarrollan. Al ser agil, es llevada a cabo por el mismo equipo y no de forma externa mediante un comite como en el tradicional.

Caracteristicas:

- SCM Responde a cambios EN LUGAR DE EVITARLOS!
- Automatizacion debe ser aplicada siempre que sea posible → Entregas frecuentes
- Retroalimentacion continua, estabilidad e integridad
- Actividades EMBEBIDAS en las demas tareas DURANTE el SPRINT
- RESPONSABILIDAD DE TODO EL EQUIPO
- Adopcion de practicas continuas

## *Continuos Integration*

Practica de desarrollo que promueve la costumbre de integrar codigo a un repositorio compartido varias veces al dia. Cada integracion es luego verificada por una serie de pruebas automatizadas con el fin de detectar problemas tempranamente.

Asegura que el SW pueda ser DESPLEGADO en cualquier momento, que compile el codigo. Se usa el TDD para esto

## *Continuos Delivery*

Disciplina que contiene a la anterior (como prerrequisito), pero agregando que el software pueda ser liberado en produccion en cualquier momento, sumando pruebas de aceptacion. Para ser liberado debe ser aceptado por algun responsable del negocio (Ejemplo, el PO). El SW debe estar siempre en un estado de “entregable”, con la build, el codigo que compila y los test aprobados. El despliegue del paquete se puede hacer en “un click”.

## *Continuous Deployment*

Encapsula a la anterior como un prerrequisito, consiste en poner en un ambiente de produccion al producto de forma automatizada, saltandose el paso humano de liberar el producto, para lo cual se utilizan pipelines que contienen una serie de pasos que deben ejecutarse para que la instalacion sea satisfactoria. Busca la mayor TRANSPARENCIA para el usuario que puso en produccion esta nueva version del producto. PELIGRO en fallas que pueden ser llevados a produccion, preferiblemente se deben hacer despliegues rapidos y cortos para poder solucionar todos los defectos.

### Canary Deployment

Consiste en realizar lanzamientos por etapas, primero se implementa una actualizacion de SW para un grupo reducido de usuarios para probarla. si es exitosa, se para a gran escala, extendiendose. Ofrecen actualizaciones sin tiempo de inactividad y reversiones sencillas, siendo mas fluidas que las Blue-Green, como tambien teniendo menor impacto en sus defectos

### Blue-Green Deployment

Modelo de lanzamiento que transfiere el trafico de usuarios entre una version anterior a una nueva version, las cuales se ejecutan en produccion. La version anterior es el BLUE, la nueva el GREEN. Una vez que se traslada todo el trafico a GREEN, el BLUE queda como failsafe o puede ser retirado directamente y se actualiza la version GREEN para que sea la estable. En sintesis, se mantienen dos entornos a diferencia del canary que hace pruebas para grupos reducidos, verificando ANTES de hacer la transicion.

# Bibliografía


- Pressman, Roger S. *Ingeniería del Software: Un enfoque práctico*.
- *Ian Somerville. Ingeniería de Software 9 edicion*
- *PMBOK Guide (Project Management Body of Knowledge). Project Management Institute.*
- *IEEE Computer Society. Guide to the Software Engineering Body of Knowledge (SWEBOK)*